

Package ‘solarr’

May 14, 2026

Type Package

Title Stochastic models for solar radiation

Version 1.0.1

Author Beniamino Sartini <beniamino.sartini2@unibo.it>.

Maintainer Beniamino Sartini <beniamino.sartini2@unibo.it>

Description Implementation of stochastic models and option pricing on solar radiation data.

Depends R (>= 4.4.0),
ggplot2,
dplyr,
mclust

Imports broom,
mixtools,
stringr,
rugarch,
purrr,
tidyr,
lubridate,
formula.tools,
numDeriv,
lifecycle

Suggests knitr,
rmarkdown,
testthat

License GPL-3

VignetteBuilder knitr

Encoding UTF-8

LazyData true

URL <https://github.com/beniamino98/solarr>

BugReports <https://github.com/beniamino98/solarr/issues>

Roxygen list(markdown = TRUE, r6 = TRUE)

RoxygenNote 7.3.3

Contents

ARMA_companion_matrix	4
ARMA_covariance	5
ARMA_expectation	6
ARMA_filter	6
ARMA_fit	7
ARMA_forecast	7
ARMA_loss_CSS	8
ARMA_loss_logLik	8
ARMA_modelR6	9
ARMA_next_step	11
ARMA_params_to_zeta	12
ARMA_variance	13
ARMA_vector_b	14
AR_params_to_phi	15
AR_params_to_zeta	15
Bologna	16
boundTransform	16
boundTransform_av_links	20
boundTransform_link.gumbel	20
boundTransform_link.identity	21
boundTransform_link.invgumbel	21
boundTransform_link.logis	21
boundTransform_link.norm	22
CAMS_data	22
clearsky_delta	23
clearsky_optimizer	23
clearsky_optimizer_CLS	24
clearsky_optimizer_delta	25
clearsky_optimizer_RLS	25
clearsky_outliers	26
control_seasonalClearsky	27
control_solarHedging	28
control_solarOption	28
desscher	29
desscherMixture	30
detect_season	31
dgumbel	32
dinvgumbel	33
dkumaraswamy	34
dmix2norm	35
dmixnorm	37
dsnrm	38
dsolarGHI	39
dsolarK	41
dsolarX	43
dsugeno	44
dtnorm	45
GARCH_companion_matrix	46
gaussianMixture	47
is_leap_year	50

ks_test	51
ks_test_ts	52
logLik_std.errors	52
mix2norm_M_Yl_given_Yv	53
mix2norm_probs_given_Yv	53
mix2norm_S_Yl_given_Yv	54
monthlyParams	54
number_of_day	55
PDF	56
PIT	57
PIT_test	57
riccati_root	57
seasonalClearsky	58
seasonalClearsky_spec	60
seasonalModel	61
seasonalSolarFunctions	64
sGARCH	71
solarHedging_model	73
solarHedging_scenarios	74
solarMixture	75
solarMixture_VaR	79
solarModel	79
solarModel_AIC_BIC	84
solarModel_calibrate_theta	84
solarModel_covariance	85
solarModel_forecast	85
solarModel_grid	86
solarModel_match_params	87
solarModel_PIT	87
solarModel_PIT_test	88
solarModel_predict	88
solarModel_predict_plot	89
solarModel_QMLE	89
solarModel_spec	90
solarModel_tests	96
solarModel_test_autocorr	97
solarModel_test_distribution	98
solarModel_test_forecast	99
solarModel_test_LPD	99
solarModel_test_pricing	99
solarModel_VaR	100
solarMoment	100
solarMoments	102
solarMoments_conditional	103
solarMoments_path	104
solarMoments_unconditional	105
solarOption	105
solarOptionPayoff	107
solarOption_calibrate_theta	108
solarOption_choquet	108
solarOption_discount	109
solarOption_greeks	110

solarOption_historical	111
solarOption_index_greeks	112
solarOption_lambda	112
solarOption_model	113
solarOption_moments	114
solarOption_payoff	115
solarOption_price	116
solarOption_pricing	117
solarOption_scenario	118
solarScenario	119
solarScenario_by	120
solarScenario_filter	121
solarScenario_plot	122
solarScenario_residuals	122
solarScenario_spec	123
solarScenario_VaR	125
solarSpectra	125
solarTransform	126
SoRadPortfolio	128
spectralDistribution	129
sugeno_bounds	129
VaR_test	130
VaR_viol	130

Index	131
--------------	------------

ARMA_companion_matrix *Matrix form of an ARMA model*

Description

[Stable] Construct the companion matrix of an ARMA model

Usage

```
ARMA_companion_matrix(phi, theta)
```

Arguments

phi	numeric vector of length p, AR parameters.
theta	numeric vector of length q, MA parameters.

Value

numeric matrix of size $p+q \times p+q$.

Note

Version 1.0.2

See Also

Other ARMA-params: [ARMA_params_to_zeta\(\)](#), [ARMA_vector_b\(\)](#), [AR_params_to_phi\(\)](#), [AR_params_to_zeta\(\)](#)

Examples

```
# AR(1) / MA(1) ~ No companion
ARMA_companion_matrix(c(0.4))
ARMA_companion_matrix(theta = c(0.4))
# Only AR
ARMA_companion_matrix(c(0.4, 0.3, 0.1))
# Only MA
ARMA_companion_matrix(theta = c(0.4, 0.3, 0.1))
# ARMA
ARMA_companion_matrix(c(0.4, 0.2), c(0.3, 0))
ARMA_companion_matrix(c(0.1, 0.02, 0.01), c(0.3, 0.1, 0.05))
```

ARMA_covariance	<i>Matrix form of an ARMA model</i>
-----------------	-------------------------------------

Description

[Stable] Compute the ARMA conditional variance / covariance

Usage

```
ARMA_covariance(h, k, A, b, sigma2 = 1)
```

Arguments

h	integer(1), number of steps ahead.
k	integer(1), number of steps ahead for the second lag.
A	numeric matrix of size p+q x p+q. See the function ARMA_companion_matrix for more details.
b	numeric vector of length p+q. See the function ARMA_vector_b for more details.
sigma2	integer(1), std. deviation of the residuals.

Note

Version 1.0.2

See Also

Other ARMA-moments: [ARMA_expectation\(\)](#), [ARMA_filter\(\)](#), [ARMA_forecast\(\)](#), [ARMA_next_step\(\)](#), [ARMA_variance\(\)](#)

Examples

```
A <- ARMA_companion_matrix(c(0.4, 0.1), c(0.1, 0.05))
b <- ARMA_vector_b(2,2)
ARMA_covariance(3, 1, A, b, sigma2 = 1)
```

ARMA_expectation	<i>Matrix form of an ARMA model</i>
------------------	-------------------------------------

Description

[Stable] Compute the conditional mean of an ARMA model

Usage

```
ARMA_expectation(h, X0, A, b, intercept = 0)
```

Arguments

<code>h</code>	integer(1), number of steps ahead.
<code>X0</code>	numeric vector of length $p + q$, state vector of past values.
<code>A</code>	numeric matrix of size $p+q \times p+q$. See the function ARMA_companion_matrix for more details.
<code>b</code>	numeric vector of length $p+q$. See the function ARMA_vector_b for more details.
<code>intercept</code>	Numeric scalar, intercept parameter.

Note

Version 1.0.2

See Also

Other ARMA-moments: [ARMA_covariance\(\)](#), [ARMA_filter\(\)](#), [ARMA_forecast\(\)](#), [ARMA_next_step\(\)](#), [ARMA_variance\(\)](#)

Examples

```
A <- ARMA_companion_matrix(c(0.4, 0.1), c(0.1, 0.05))
b <- ARMA_vector_b(2,2)
intercept <- 0.2
X0 <- c(0.9, 0.2, -0.1, 0.3)
ARMA_expectation(h = 3, X0, A, b, intercept)
```

ARMA_filter	<i>Matrix form of an ARMA model</i>
-------------	-------------------------------------

Description

[Stable] Filter the time-series and compute fitted values and residuals.

Usage

```
ARMA_filter(x, A, b, intercept = 0)
```

Arguments

x	numeric vector, time series to filter.
A	numeric matrix of size $p+q \times p+q$. See the function ARMA_companion_matrix for more details.
b	numeric vector of length $p+q$. See the function ARMA_vector_b for more details.
intercept	Numeric scalar, intercept parameter.

Note

Version 1.0.2

See Also

Other ARMA-moments: [ARMA_covariance\(\)](#), [ARMA_expectation\(\)](#), [ARMA_forecast\(\)](#), [ARMA_next_step\(\)](#), [ARMA_variance\(\)](#)

ARMA_fit	<i>Fit with constraint parameters</i>
----------	---------------------------------------

Description

Fit with constraint parameters

Usage

```
ARMA_fit(y, arOrder = 1, maOrder = 0, method = c("CSS", "ML"))
```

ARMA_forecast	<i>Matrix form of an ARMA model</i>
---------------	-------------------------------------

Description

[Stable] Fast ARMA state-space h -step forecast and weights (C/BLAS)

Usage

```
ARMA_forecast(h, X0, A, b, intercept = 0)
```

Arguments

h	integer(1), number of steps ahead.
X0	numeric vector of length $p + q$, state vector of past values.
A	numeric matrix of size $p+q \times p+q$. See the function ARMA_companion_matrix for more details.
b	numeric vector of length $p+q$. See the function ARMA_vector_b for more details.
intercept	Numeric scalar, intercept parameter.

Note

Version 1.0.2

See Also

Other ARMA-moments: [ARMA_covariance\(\)](#), [ARMA_expectation\(\)](#), [ARMA_filter\(\)](#), [ARMA_next_step\(\)](#), [ARMA_variance\(\)](#)

Examples

```
h <- 1000
X0 <- c(0.2, 0.1)
A <- ARMA_companion_matrix(0.2, 0.1)
b <- ARMA_vector_b(1,1)
ARMA_forecast(h, X0, A, b, intercept = 0)
```

ARMA_loss_CSS

Sum of the errors with constraint parameters

Description

Sum of the errors with constraint parameters

Usage

```
ARMA_loss_CSS(params, p = 0, q = 0, y)
```

ARMA_loss_logLik

Log-likelihood with constraint parameters

Description

Log-likelihood with constraint parameters

Usage

```
ARMA_loss_logLik(params, p = 0, q = 0, y, per_obs = FALSE)
```


ARMA_modelR6

*ARMA model***Description**

R6 class for ARMA(p, q) model

Public fields

control list, to contain custom control parameters.

model_name character(1), standard model name.

Active bindings

arOrder integer(1), Autoregressive order.

maOrder integer(1), Moving-Average order.

order named numeric vector of size 2. Orders of the ARMA model: the first element is the arOrder, while the second the maOrder.

intercept named integer(1), intercept of the model.

phi named numeric vector of size arOrder, AR parameters. If arOrder = 0, the parameter is zero.

theta named numeric vector of size maOrder, MA parameters. If maOrder = 0, the parameter is zero.

coefficients named numeric vector of size arOrder + maOrder + 1. The first element is the intercept, then the ARMA parameters.

std.errors Numeric named vector, standard errors of the intercept and ARMA parameters.

sigma2 integer(1), std.errors of the residuals.

A numeric matrix of size (arOrder + maOrder) x (arOrder + maOrder). See the function [ARMA_companion_matrix](#) for more details.

b numeric vector of size arOrder + maOrder. See the function [ARMA_vector_b](#) for more details.

tidy Tibble with estimated parameters and standard errors.

Methods**Public methods:**

- [ARMA_modelR6\\$new\(\)](#)
- [ARMA_modelR6\\$fit\(\)](#)
- [ARMA_modelR6\\$filter\(\)](#)
- [ARMA_modelR6\\$next_step\(\)](#)
- [ARMA_modelR6\\$expectation\(\)](#)
- [ARMA_modelR6\\$variance\(\)](#)
- [ARMA_modelR6\\$update\(\)](#)
- [ARMA_modelR6\\$update_std.errors\(\)](#)
- [ARMA_modelR6\\$update_sigma2\(\)](#)
- [ARMA_modelR6\\$print\(\)](#)
- [ARMA_modelR6\\$clone\(\)](#)

Method `new()`: Initialize an ARMA model

Usage:

```
ARMA_modelR6$new(arOrder = 1, maOrder = 1, include.intercept = FALSE)
```

Arguments:

`arOrder` integer(1), order of the Autoregressive component.

`maOrder` integer(1), order of the Moving-Average component.

`include.intercept` logical(1), the default is FALSE. When TRUE the intercept will be included.

Method `fit()`: Fit the ARMA model with `arma` function.

Usage:

```
ARMA_modelR6$fit(x)
```

Arguments:

`x` numeric vector, time series to fit.

Method `filter()`: Filter the time-series and compute fitted values and residuals. See the function [ARMA_filter](#) for more details.

Usage:

```
ARMA_modelR6$filter(x)
```

Arguments:

`x` numeric vector, time series to filter.

Method `next_step()`: Next step function. See the function [ARMA_next_step](#) for more details.

Usage:

```
ARMA_modelR6$next_step(x, n.ahead = 1, eps = 0)
```

Arguments:

`x` Numeric vector, state vector with past observations and residuals.

`n.ahead` integer(1), number of steps ahead.

`eps` optional numeric vector of length `n.ahead`, next step realized residuals.

Method `expectation()`: Forecast expected value. See the function [ARMA_expectation](#) for more details.

Usage:

```
ARMA_modelR6$expectation(h = 1, X0)
```

Arguments:

`h` integer(1), number of steps ahead.

`X0` numeric vector of size `p + q`. State vector of past values.

Method `variance()`: Forecast variance. See the function [ARMA_variance](#) for more details.

Usage:

```
ARMA_modelR6$variance(h = 1, sigma2 = 1)
```

Arguments:

`h` integer(1), number of steps ahead.

`sigma2` integer(1), standard deviation of the residuals.

Method `update()`: Update the model's parameters

Usage:

```
ARMA_modelR6$update(coefficients)
```

Arguments:

`coefficients` Numeric named vector, model's coefficients. If missing nothing will be updated.

Method `update_std.errors()`: Update the standard errors of the parameters.

Usage:

```
ARMA_modelR6$update_std.errors(std.errors)
```

Arguments:

`std.errors` Numeric named vector, parameters' standard errors. If missing nothing will be updated.

Method `update_sigma2()`: Update the variance of the residuals.

Usage:

```
ARMA_modelR6$update_sigma2(sigma2)
```

Arguments:

`sigma2` integer(1), variance of the residuals.

Method `print()`: Print method for ARMA_modelR6 class.

Usage:

```
ARMA_modelR6$print()
```

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
ARMA_modelR6$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Note

Version 1.0.2

See Also

[stats::arima\(\)](#) which is wrapped in the method `fit`.

ARMA_next_step

Matrix form of an ARMA model

Description

[Stable] Next-step value of an ARMA process

Usage

```
ARMA_next_step(h = 1, X0, A, b, intercept = 0, eps = 0)
```

Arguments

<code>h</code>	integer(1), number of steps ahead.
<code>X0</code>	numeric vector of length $p + q$, state vector of past values.
<code>A</code>	numeric matrix of size $p+q \times p+q$. See the function ARMA_companion_matrix for more details.
<code>b</code>	numeric vector of length $p+q$. See the function ARMA_vector_b for more details.
<code>intercept</code>	Numeric scalar, intercept parameter.
<code>eps</code>	optional numeric vector of length h . Next step new residuals.

Note

Version 1.0.2

See Also

Other ARMA-moments: [ARMA_covariance\(\)](#), [ARMA_expectation\(\)](#), [ARMA_filter\(\)](#), [ARMA_forecast\(\)](#), [ARMA_variance\(\)](#)

Examples

```
# Companion matrix and vector b
A <- ARMA_companion_matrix(c(0.4), c())
b <- ARMA_vector_b(1,0)
# Initial value
X0 <- c(0.9)

# Next step forecast
ARMA_next_step(1, X0, A, b, intercept = 0.2, eps = 0)
# 3-steps ahead forecast
ARMA_next_step(3, X0, A, b, intercept = 0.2, eps = 0)
# 10-step ahead forecast
ARMA_next_step(10, X0, A, b, intercept = 0.2, eps = 0)

# Next step simulation
eps <- rnorm(1)
ARMA_next_step(1, X0, A, b, intercept = 0.2, eps = eps)
# 3-steps ahead simulation
eps <- rnorm(3)
ARMA_next_step(3, X0, A, b, intercept = 0.2, eps = eps)
# 10-step ahead simulation
eps <- rnorm(10)
ARMA_next_step(10, X0, A, b, intercept = 0.2, eps = eps)
```

ARMA_params_to_zeta *ARMA reparametrization*

Description

[Stable] Map constraint ARMA parameters into unconstraint parameters

Usage

```
ARMA_params_to_zeta(phi, theta)
```

Arguments

zeta_phi Numeric vector of length p, AR parameters.
 zeta_theta Numeric vector of length q, MA parameters.

Note

Version 1.0.2

See Also

Other ARMA-params: [ARMA_companion_matrix\(\)](#), [ARMA_vector_b\(\)](#), [AR_params_to_phi\(\)](#), [AR_params_to_zeta\(\)](#)

Examples

```
phi <- c(0.3, 0.4, 0.1)
theta <- c(0.1, 0.8)
zeta <- ARMA_params_to_zeta(phi, theta)
```

ARMA_variance	<i>Matrix form of an ARMA model</i>
---------------	-------------------------------------

Description

[[Stable](#)] Compute the variance of an ARMA model

Usage

```
ARMA_variance(h = 1, A, b, sigma2 = 1)
```

Arguments

h integer(1), number of steps ahead.
 A numeric matrix of size p+q x p+q. See the function [ARMA_companion_matrix](#) for more details.
 b numeric vector of length p+q. See the function [ARMA_vector_b](#) for more details.
 sigma2 integer(1), std. deviation of the residuals.

Note

Version 1.0.2

See Also

Other ARMA-moments: [ARMA_covariance\(\)](#), [ARMA_expectation\(\)](#), [ARMA_filter\(\)](#), [ARMA_forecast\(\)](#), [ARMA_next_step\(\)](#)

Examples

```
A <- ARMA_companion_matrix(c(0.4, 0.1), c(0.1, 0.05))
b <- ARMA_vector_b(2,2)
ARMA_variance(h = 3, A, b, sigma2 = 1)
```

ARMA_vector_b	<i>Matrix form of an ARMA model</i>
---------------	-------------------------------------

Description

[Stable] Construct the vector b of an ARMA model

Usage

```
ARMA_vector_b(arOrder, maOrder)
```

Arguments

arOrder	integer(1), order of AR model.
maOrder	integer(1), order of MA model.

Note

Version 1.0.2

See Also

Other ARMA-params: [ARMA_companion_matrix\(\)](#), [ARMA_params_to_zeta\(\)](#), [AR_params_to_phi\(\)](#), [AR_params_to_zeta\(\)](#)

Examples

```
# Vector for ARMA(2,2)
ARMA_vector_b(2,2)
# Vector for AR(2)
ARMA_vector_b(2,0)
# Vector for MA(2)
ARMA_vector_b(0,2)
```

AR_params_to_phi	AR reparametrization
------------------	----------------------

Description

[Stable] Map unconstraint AR parameters into constraint parameters

Usage

```
AR_params_to_phi(zeta)
```

Arguments

zeta Numeric vector of length p, unconstraint parameters.

Note

Version 1.0.2

See Also

Other ARMA-params: [ARMA_companion_matrix\(\)](#), [ARMA_params_to_zeta\(\)](#), [ARMA_vector_b\(\)](#), [AR_params_to_zeta\(\)](#)

AR_params_to_zeta	Unconstraint parameters for AR
-------------------	--------------------------------

Description

[Stable] Map AR parameters into unconstraint parameters

Usage

```
AR_params_to_zeta(phi)
```

Arguments

phi numeric vector of length p, AR parameters.

Note

Version 1.0.2

See Also

Other ARMA-params: [ARMA_companion_matrix\(\)](#), [ARMA_params_to_zeta\(\)](#), [ARMA_vector_b\(\)](#), [AR_params_to_phi\(\)](#)

Bologna	<i>Bologna model specification</i>
---------	------------------------------------

Description

A saved solarModel_spec object for Bologna. The data file Bologna.RData loads an object named spec.

Usage

```
data(Bologna)
```

Format

An R6 object of class solarModel_spec.

Details

The object is included as an example package specification. It has place set to "Bologna" and target set to "GHI".

boundTransform	<i>Bounded transformation functions</i>
----------------	---

Description

Bounded transformation functions

Bounded transformation functions

Public fields

epsilon Numeric, ϵ transformation parameter.

Active bindings

alpha Numeric, α transformation parameter.

beta Numeric, β transformation parameter.

link Character, name of the link function g .

monotonicity Character, type of monotonicity of g .

Methods

Public methods:

- `boundTransform$new()`
- `boundTransform$set_transform()`
- `boundTransform$X_prime()`
- `boundTransform$iX_prime()`
- `boundTransform$Y()`
- `boundTransform$iY()`
- `boundTransform$g()`
- `boundTransform$ig()`
- `boundTransform$g_prime()`
- `boundTransform$fit()`
- `boundTransform$bounds()`
- `boundTransform$update()`
- `boundTransform$print()`
- `boundTransform$clone()`

Method `new()`: Initialize a `boundTransform` object.

Usage:

```
boundTransform$new(alpha = 0, beta = 1, link = "invgumbel")
```

Arguments:

`alpha` Numeric, α transformation parameter.

`beta` Numeric, β transformation parameter.

`link` Character, link function.

Method `set_transform()`: Set the transform function g, g^{-1}, g' .

Usage:

```
boundTransform$set_transform(link)
```

Arguments:

`link` Character, link function. Valid links are "invgumbel", "gumbel", "logis", "norm".

Method `X_prime()`: Map the risk-driver X_t into the normalized risk-driver X'_t .

Usage:

```
boundTransform$X_prime(Xt)
```

Arguments:

`Xt` Numeric, risk driver in $X_t \in (\alpha, \alpha + \beta)$.

Details: The function computes:

$$X'(X_t) = \frac{X_t - \alpha}{\beta}$$

Returns: Numeric, normalized risk driver $X'_t \in (0, 1)$.

Method `iX_prime()`: Map the normalized variable X'_t to the risk driver X_t .

Usage:

```
boundTransform$iX_prime(Xt_prime)
```

Arguments:

`Xt_prime` Numeric, normalized risk driver $X'_t \in (0, 1)$.

Details: The function computes:

$$iX'(X'_t) = \alpha + \beta \cdot X'_t$$

Returns: Numeric, risk driver in $X_t \in (\alpha, \alpha + \beta)$.

Method `Y()`: Map the normalized risk driver X'_t in the transformed variable Y_t

Usage:

`boundTransform$Y(Xt_prime)`

Arguments:

`Xt_prime` Numeric, normalized risk driver $X'_t \in (0, 1)$.

Details: The function computes:

$$Y(X'_t) = g(X'_t)$$

Returns: Numeric, transformed variable $Y_t \in (-\infty, \infty)$.

Method `iY()`: Map the transformed variable Y_t into the normalized risk driver X'_t

Usage:

`boundTransform$iY(Yt)`

Arguments:

`Yt` Numeric, transformed variable $Y_t \in (-\infty, \infty)$.

Details: The function computes:

$$iY(Y_t) = g^{-1}(Y_t)$$

Returns: Numeric, normalized risk driver $X'_t \in (0, 1)$.

Method `g()`: Link function to map X'_t to Y_t .

Usage:

`boundTransform$g(X_prime)`

Arguments:

`X_prime` Numeric, normalized risk driver $X'_t \in (0, 1)$.

Returns: Numeric, transformed variable $Y_t \in (-\infty, \infty)$.

Method `ig()`: Inverse of the function to map Y_t to X' .

Usage:

`boundTransform$ig(Yt)`

Arguments:

`Yt` Numeric, transformed variable $Y_t \in (-\infty, \infty)$.

Returns: Numeric, normalized risk driver $X'_t \in (0, 1)$.

Method `g_prime()`: First derivative of the function g .

Usage:

`boundTransform$g_prime(X_prime)`

Arguments:

`X_prime` Numeric, normalized risk driver $X'_t \in (0, 1)$.

Method `fit()`: Fit the best parameters α and β from a given time series

Usage:

```
boundTransform$fit(x, epsilon = 0.01, min_pos = 1, max_pos = 1)
```

Arguments:

`x` time series of solar risk drivers in $(0, 1)$.

`epsilon` Numeric

`min_pos` Integer, position of the minimum. For example when 2 the minimum is the second lowest value.

`max_pos` Integer, position of the maximum. For example when 3 the maximum is the third greatest value.

Details: Return a list that contains:

alpha Numeric, α transformation parameter.

beta Numeric, β transformation parameter.

epsilon Numeric, threshold used for fitting.

Xt_min Numeric, minimum value of the time series.

Xt_max Numeric, maximum value of the time series.

Returns: A named list.

Method `bounds()`: Compute the bounds for the transformed variables.

Usage:

```
boundTransform$bounds(target = "Xt")
```

Arguments:

`target` target variable. Available choices are:

"Xt" Solar risk driver, the bounds returned are $[\alpha, \alpha + \beta]$.

"Kt" Clearness index, the bounds returned are $[1 - \alpha - \beta, 1 - \alpha]$.

"Yt" Solar transform, the bounds returned are $[-\infty, \infty]$.

Returns: A numeric vector where the first element is the lower bound and the second the upper bound.

Method `update()`: Update the transformation parameters α and β .

Usage:

```
boundTransform$update(alpha, beta)
```

Arguments:

`alpha` Numeric, transformation parameter.

`beta` Numeric, transformation parameter.

Returns: Update the slots `$alpha` and `$beta`.

Method `print()`: Print method for the class `boundTransform`

Usage:

```
boundTransform$print()
```

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
boundTransform$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Note

Version 1.0.3

Examples

```
st <- boundTransform$new()
```

```
boundTransform_av_links
```

List all the available link functions

Description

List all the available link functions

Usage

```
boundTransform_av_links()
```

Note

Version 1.0.0

Examples

```
boundTransform_av_links()
```

```
boundTransform_link.gumbel
```

Link function (Gumbel)

Description

Link function (Gumbel)

Usage

```
boundTransform_link.gumbel()
```

Note

Version 1.0.0

`boundTransform_link.identity`
Link function (identity)

Description

Link function (identity)

Usage

`boundTransform_link.identity()`

Note

Version 1.0.0

`boundTransform_link.invgumbel`
Link function (invGumbel)

Description

Link function (invGumbel)

Usage

`boundTransform_link.invgumbel()`

Note

Version 1.0.0

`boundTransform_link.logis`
Link function (logit)

Description

Link function (logit)

Usage

`boundTransform_link.logis()`

Note

Version 1.0.0

```
boundTransform_link.norm
```

Link function (probit)

Description

Link function (probit)

Usage

```
boundTransform_link.norm()
```

Note

Version 1.0.0

```
CAMS_data
```

CAMS data by location

Description

A list of daily solar radiation data sets for selected locations.

Usage

```
data(CAMS_data)
```

Format

A named list. Each element is a tibble with 7,465 rows and 8 columns:

date Date.

n Numeric day index.

Year Numeric year.

Month Numeric month.

Day Integer day of month.

GHI Numeric global horizontal irradiance value.

clearsky Numeric clear-sky radiation value.

H0 Numeric extraterrestrial radiation value.

Details

The list contains named elements for the locations available to `solarModel_spec$specification()`. Individual elements include `place` and `coords` attributes.

clearsky_delta	<i>Optimizer for Solar Clear sky</i>
----------------	--------------------------------------

Description

Find the best parameter delta for fitting clear sky radiation.

Usage

```
clearsky_delta(x, Ct, lower = 0, upper = 3, by = 0.01, ntol = 0)
```

Arguments

x	Numeric vector, time series of solar radiation.
Ct	Numeric vector, time series of clear sky radiation.
lower	Numeric scalar, lower bound for grid of delta parameters used for optimization. Default is 0.
upper	Numeric scalar, upper bound for grid of delta parameters used for optimization. Default is 3.
by	Numeric scalar, step for the grid. Default is 0.01.
ntol	Integer scalar, Tolerance for the maximum number of violations admitted of the condition $Ct > x$. The default is 0.

Details

Detect the best parameter delta such that the condition $x > \text{delta} * Ct$ is satisfied for all the observations except for ntol observations.

Note

Version 1.0.1

clearsky_optimizer	<i>Fit the seasonal model for clear sky radiation.</i>
--------------------	--

Description

Fit the seasonal model for clear sky radiation.

Usage

```
clearsky_optimizer(
  x,
  dates,
  clearsky,
  lat,
  spec = seasonalClearsky_spec(),
  control = control_seasonalClearsky(),
  seasonal_model_Ct
)
```

Arguments

x	Numeric vector, time series of solar radiation.
dates	Character or Date vector, time series of dates.
clearsky	Numeric vector, time series of target clear sky radiation.
lat	Numeric scalar, reference latitude. Overwritten when seasonal_model_Ct is specified.
spec	See the function seasonalClearsky_spec for more details. Overwritten when seasonal_model_Ct is specified.
control	See the function control_seasonalClearsky for more details. Overwritten when seasonal_model_Ct is specified.
seasonal_model_Ct	Optional object of the class seasonalClearsky . When submitted lat, spec and control parameters used are recovered from the specified model.

Note

Version 1.0.0

Examples

```
x = solarr::spec$data$GHI
dates = solarr::spec$data$date
lat = solarr::spec$coords$lat
clearsky = solarr::spec$data$clearsky
# Without a clearsky model
seasonal_model_Ct <- clearsky_optimizer(x, dates, clearsky, lat)
# With a clearsky model
clearsky_optimizer(x, dates, clearsky, seasonal_model_Ct = seasonal_model_Ct)
```

clearsky_optimizer_CLS

Fully constrained optimization (Constrained Least Squares)

Description

Fully constrained optimization (Constrained Least Squares)

Usage

```
clearsky_optimizer_CLS(
  seasonal_model_Ct,
  newdata,
  eps = 0.01,
  delta_optimize = TRUE
)
```


Arguments

seasonal_model_Ct	An object of the class seasonalClearsky. See seasonalClearsky for more details.
newdata	dataset
eps	threshold parameter

Note

Version 1.0.0

clearsky_optimizer_delta

Delta optimizer on a clear-sky model

Description

Delta optimizer on a clear-sky model

Usage

```
clearsky_optimizer_delta(seasonal_model_Ct, newdata)
```

Arguments

seasonal_model_Ct	An object of the class seasonalClearsky. See seasonalClearsky for more details.
newdata	dataset

Note

Version 1.0.0

clearsky_optimizer_RLS

Penalized Least-Squares

Description

Penalized Least-Squares

Usage

```
clearsky_optimizer_RLS(  
  seasonal_model_Ct,  
  newdata,  
  lambda = 0,  
  delta_optimize = TRUE  
)
```

Arguments

seasonal_model_Ct	An object of the class seasonalClearsky. See seasonalClearsky for more details.
newdata	dataset
lambda	penalization parameter.

Note

Version 1.0.0

clearsky_outliers	<i>Impute clear sky outliers</i>
-------------------	----------------------------------

Description

Detect and impute outliers with respect to a maximum level of radiation (Ct)

Usage

```
clearsky_outliers(x, Ct, date, threshold = 1e-04, quiet = FALSE)
```

Arguments

x	Numeric vector, time series of solar radiation.
Ct	Numeric vector, time series of fitted clear sky radiation.
date	Character or Date vector, time series of dates used to precisely impute solar radiation according to the realized values in the same day of the year.
threshold	Numeric, scalar, threshold value used for imputation. Default is 0.0001.
quiet	Logical.

Details

The function will detect the observations for which $x > Ct$, $x < 0$ or $is.na(x)$. Then, if

$x < 0$ If a value is below 0 for a day it will be imputed to be equal to $\min(x)$ for that day.

$x > Ct$ If a value is above the maximum clear sky Ct it will be imputed to be $Ct \cdot (1 - \text{threshold})$.

$is.na(x)$ If a value is NA it will be imputed to be the average $\text{mean}(x)$ for that day..

Note

Version 1.0.0

Examples

```
clearsky_outliers(c(1,2,3), 2)
```

control_seasonalClearsky

Control parameters for a seasonalClearsky object

Description

Control parameters for a seasonalClearsky object

Usage

```
control_seasonalClearsky(
  delta0 = 1.4,
  lower = 0,
  upper = 3,
  by = 0.001,
  ntol = 0,
  eps = 0.01,
  lambda = 100,
  quiet = FALSE
)
```

Arguments

delta0	Numeric scalar, initial value for the optimization. The estimated clear sky is increased by delta0.
lower	Numeric scalar, lower bound for grid of delta parameters used for optimization. Default is 0.
upper	Numeric scalar, upper bound for grid of delta parameters used for optimization. Default is 3.
by	Numeric scalar, step for the grid. Default is 0.01.
ntol	Integer scalar, Tolerance for the maximum number of violations admitted of the condition $C_t > x$. The default is 0.
eps	threshold parameter
lambda	penalization parameter.
quiet	Logical, when FALSE, the default, the functions displays warning or messages.

Details

The parameters ntol, lower, upper and by are used exclusively in [clearsky_delta](#).

Value

Named list of control parameters.

Note

Version 1.0.1

Examples

```
control = control_seasonalClearsky()
```

control_solarHedging *Control parameters for solar hedging*

Description

Control parameters for solar hedging

Usage

```
control_solarHedging(
  n_panels = 1,
  efficiency = 1,
  PUN = 1,
  tick = 1,
  n_contracts = 1
)
```

Arguments

n_panels	Numeric scalar, number of meters squared of solar panels.
efficiency	Numeric scalar, average electricity produced with 1 m^2 of solar panels given 1 kWh/ m^2 of solar radiation received.
PUN	Numeric scalar, fixed electricity price at which the produced energy is sold.
tick	Numeric scalar, tick for the monetary conversion of the payoff of a solar derivative from kWh/ m^2 to Euros.
n_contract	Numeric scalar, number of solar derivative contracts bought.

Note

Version 1.0.0.

control_solarOption *Control parameters for a solar option*

Description

Control parameters for a solar option

Usage

```
control_solarOption(
  nyears = c(2005, 2025),
  K = 0,
  leap_year = FALSE,
  nsim = 200,
  ci = 0.05,
  seed = 1
)
```

Arguments

nyears	numeric vector. Interval of years considered. The first element will be the minimum and the second the maximum years used in the computation of the fair payoff.
K	numeric, level for the strike with respect to the seasonal mean. The seasonal mean is multiplied by $\exp(K)$.
leap_year	logical, when FALSE, the default, the year will be considered of 365 days, otherwise 366.
nsim	integer, number of simulations used to bootstrap the premium's bounds. See solarOption_historical_bootstrap .
ci	numeric, confidence interval for bootstrapping. See solarOption_historical_bootstrap .
seed	integer, random seed for reproducibility. See solarOption_historical_bootstrap .

Note

Version 1.0.0.

Examples

```
control_options <- control_solarOption()
```

desscher

Esscher-distorted density and distribution

Description

Given a function of x , i.e. $f_X(x)$, compute its Esscher transform and return again a function of x .

Usage

```
desscher(pdf, theta = 0, lower = -Inf, upper = Inf)
```

```
pesscher(pdf, theta = 0, lower = -Inf, upper = Inf)
```

Arguments

pdf	Function, density function to distort.
theta	Numeric, distortion parameter.
lower, upper	Numeric, lower and upper bounds for integration, i.e. the bounds of the pdf.

Details

Given a pdf $f_X(x)$ the function computes its Esscher transform, i.e.

$$\mathcal{E}_\theta\{f_X(x)\} = \frac{e^{\theta x} f_X(x)}{\int_{-\infty}^{\infty} e^{\theta x} f_X(x) dx}$$

Value

A function of x.

Note

Version 1.0.0.

See Also

Other distributions: [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnorm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
# Grid of points
grid <- seq(-3, 3, 0.1)
# Density function of x
pdf <- function(x) dnorm(x, mean = 0)
# Esscher density (no transform)
esscher_pdf <- desscher(pdf, theta = 0)
pdf(grid) - esscher_pdf(grid)
# Esscher density (transform)
esscher_pdf_1 <- function(x) dnorm(x, mean = -0.1)
esscher_pdf_2 <- desscher(pdf, theta = -0.1)
esscher_pdf_1(grid) - esscher_pdf_2(grid)

# Esscher Distribution (transform)
esscher_cdf <- pesscher(pdf, theta = -0.1)
plot(esscher_cdf(grid))
```

desscherMixture

Esscher-distorted density and distribution of a Gaussian Mixture

Description

Esscher-distorted density and distribution of a Gaussian Mixture

Usage

```
desscherMixture(mean = c(0, 0), sd = c(1, 1), alpha = c(0.5, 0.5), theta = 0)
```

```
pesscherMixture(mean = c(0, 0), sd = c(1, 1), alpha = c(0.5, 0.5), theta = 0)
```

Arguments

mean	Numeric vector of component means.
sd	Numeric vector of component standard deviations.
alpha	Numeric vector of component probabilities.
theta	Numeric, distortion parameter.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
library(ggplot2)
grid <- seq(-5, 5, 0.01)
# Density
pdf_1 <- desscherMixture(mean = c(-3, 3), theta = 0)(grid)
pdf_2 <- desscherMixture(mean = c(-3, 3), theta = -0.5)(grid)
pdf_3 <- desscherMixture(mean = c(-3, 3), theta = 0.5)(grid)
ggplot()+
  geom_line(aes(grid, pdf_1), color = "black")+
  geom_line(aes(grid, pdf_2), color = "green")+
  geom_line(aes(grid, pdf_3), color = "red")
# Distribution
cdf_1 <- pesscherMixture(mean = c(-3, 3), theta = 0)(grid)
cdf_2 <- pesscherMixture(mean = c(-3, 3), theta = -0.2)(grid)
cdf_3 <- pesscherMixture(mean = c(-3, 3), theta = 0.2)(grid)
ggplot()+
  geom_line(aes(grid, cdf_1), color = "black")+
  geom_line(aes(grid, cdf_2), color = "green")+
  geom_line(aes(grid, cdf_3), color = "red")
```

detect_season

Detect the season

Description

Detect the season from a vector of dates

Usage

```
detect_season(x, invert = FALSE)
```

Arguments

x vector of dates in the format YYYY-MM-DD.

invert logical, when TRUE the seasons will be inverted.

Value

a character vector containing the correspondent season. Can be spring, summer, autumn, winter.

Examples

```
detect_season("2040-01-31")
detect_season(c("2040-01-31", "2023-04-01", "2015-09-02"))
```

dgumbel	<i>Gumbel distribution</i>
---------	----------------------------

Description

Density, distribution function, quantile function, and random generation for the Gumbel distribution.

Usage

```
dgumbel(x, location = 0, scale = 1, log = FALSE)

pgumbel(q, location = 0, scale = 1, log.p = FALSE, lower.tail = TRUE)

qgumbel(p, location = 0, scale = 1, log.p = FALSE, lower.tail = TRUE)

rgumbel(n, location = 0, scale = 1)
```

Arguments

x	Numeric vector of quantiles.
location	Numeric location parameter.
scale	Numeric scale parameter.
log	Logical. If TRUE, dgumbel() returns log-densities.
q	Numeric vector of quantiles.
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
p	Numeric vector of probabilities.
n	Number of observations.

Value

- dgumbel() returns a numeric vector of density values.
- pgumbel() returns a numeric vector of probabilities.
- qgumbel() returns a numeric vector of quantiles.
- rgumbel() returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
dgumbel(c(-1, 0, 1), location = 0, scale = 1)
pgumbel(c(-1, 0, 1), location = 0, scale = 1)
qgumbel(c(0.25, 0.75), location = 0, scale = 1)

set.seed(1)
rgumbel(3, location = 0, scale = 1)
```

dinvgumbel	<i>Inverted Gumbel distribution</i>
------------	-------------------------------------

Description

Density, distribution function, quantile function, and random generation for the inverted Gumbel distribution.

Usage

```
dinvgumbel(x, location = 0, scale = 1, log = FALSE)

pinvgumbel(q, location = 0, scale = 1, log.p = FALSE, lower.tail = TRUE)

qinvgumbel(p, location = 0, scale = 1, log.p = FALSE, lower.tail = TRUE)

rinvgumbel(n, location = 0, scale = 1)
```

Arguments

x	Numeric vector of quantiles.
location	Numeric location parameter.
scale	Numeric scale parameter.
log	Logical. If TRUE, <code>dinvgumbel()</code> returns log-densities.
q	Numeric vector of quantiles.
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
p	Numeric vector of probabilities.
n	Number of observations.

Value

- `dinvgumbel()` returns a numeric vector of density values.
- `pinvgumbel()` returns a numeric vector of probabilities.
- `qinvgumbel()` returns a numeric vector of quantiles.
- `rinvgumbel()` returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
dinvgumbel(c(-1, 0, 1), location = 0, scale = 1)
pinvgumbel(c(-1, 0, 1), location = 0, scale = 1)
qinvgumbel(c(0.25, 0.75), location = 0, scale = 1)

set.seed(1)
rinvgumbel(3, location = 0, scale = 1)
```

dkumaraswamy	<i>Kumaraswamy distribution</i>
--------------	---------------------------------

Description

Density, distribution function, quantile function, and random generation for the Kumaraswamy distribution.

Usage

```
dkumaraswamy(x, a = 1, b = 1, log = FALSE)

pkumaraswamy(q, a = 1, b = 1, log.p = FALSE, lower.tail = TRUE)

qkumaraswamy(p, a = 1, b = 1, log.p = FALSE, lower.tail = TRUE)

rkumaraswamy(n, a = 1, b = 1)
```

Arguments

x	Numeric vector of quantiles.
a	Numeric shape parameter. Must be positive.
b	Numeric shape parameter. Must be positive.
log	Logical. If TRUE, dkumaraswamy() returns log-densities.
q	Numeric vector of quantiles.
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
p	Numeric vector of probabilities.
n	Number of observations.

Value

- `dkumaraswamy()` returns a numeric vector of density values.
- `pkumaraswamy()` returns a numeric vector of probabilities.
- `qkumaraswamy()` returns a numeric vector of quantiles.
- `rkumaraswamy()` returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
dkumaraswamy(c(0.25, 0.5), a = 2, b = 1.5)
pkumaraswamy(c(0.25, 0.5), a = 2, b = 1.5)
qkumaraswamy(c(0.25, 0.75), a = 2, b = 1.5)

set.seed(1)
rkumaraswamy(3, a = 2, b = 1.5)
```

dmix2norm

Bivariate Gaussian mixture distribution

Description

Density, distribution function, and directional quantile approximation for a bivariate Gaussian mixture with four joint states.

Usage

```
dmix2norm(
  x,
  means = matrix(0, 4, 2),
  Sigma = rep(list(diag(1, 2)), 4),
  probs = rep(1/4, 4),
  log = FALSE
)

pmix2norm(
  x,
  means = matrix(0, 4, 2),
  Sigma = rep(list(diag(1, 2)), 4),
  probs = rep(1/4, 4),
  log = FALSE
)

qmix2norm(
```

```

p,
means = matrix(0, 4, 2),
Sigma = rep(list(diag(1, 2)), 4),
probs = rep(1/4, 4),
v = NULL,
x0 = c(-1, 1),
range_s = c(-100, 100),
log.p = FALSE,
lower.tail = TRUE
)

```

Arguments

<code>x</code>	Numeric matrix. Rows are observations and columns are the two locations/components.
<code>means</code>	Numeric matrix of component means.
<code>Sigma</code>	List of variance-covariance matrices.
<code>probs</code>	Numeric vector of joint state probabilities.
<code>log</code>	Logical. If TRUE, <code>dmix2norm()</code> or <code>pmix2norm()</code> returns values on the log scale.
<code>p</code>	Numeric vector of probabilities.
<code>v</code>	Optional numeric direction vector used by <code>qmix2norm()</code> .
<code>x0</code>	Numeric reference point used by <code>qmix2norm()</code> .
<code>range_s</code>	Numeric vector of length two giving the search interval for the scalar root used by <code>qmix2norm()</code> .
<code>log.p</code>	Logical. If TRUE, probabilities are supplied on the log scale.
<code>lower.tail</code>	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.

Details

Version 1.0.0.

Value

- `dmix2norm()` returns a numeric vector of density values.
- `pmix2norm()` returns a numeric vector of probabilities.
- `qmix2norm()` returns a numeric matrix of directional quantiles.

Examples

```

x <- matrix(c(-1, 0, 0, 1), ncol = 2, byrow = TRUE)
dmix2norm(x)
pmix2norm(x)
qmix2norm(c(0.25, 0.75))

```

dmixnorm

*Gaussian mixture distribution***Description**

[Stable] Density, distribution function, quantile function, and random generation for a univariate Gaussian mixture.

Usage

```
dmixnorm(x, mean = rep(0, 2), sd = rep(1, 2), alpha = rep(1/2, 2), log = FALSE)
```

```
pmixnorm(
  q,
  mean = rep(0, 2),
  sd = rep(1, 2),
  alpha = rep(1/2, 2),
  lower.tail = TRUE,
  log.p = FALSE
)
```

```
qmixnorm(
  p,
  mean = rep(0, 2),
  sd = rep(1, 2),
  alpha = rep(1/2, 2),
  lower.tail = TRUE,
  log.p = FALSE
)
```

```
rmixnorm(n, mean = rep(0, 3), sd = rep(1, 3), alpha = rep(1/3, 3))
```

Arguments

x	Numeric vector of quantiles.
mean	Numeric vector of component means.
sd	Numeric vector of component standard deviations.
alpha	Numeric vector of component probabilities.
log	Logical. If TRUE, dmixnorm() returns log-densities.
q	Numeric vector of quantiles.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
p	Numeric vector of probabilities.
n	Number of observations.

Value

- `dmixnorm()` returns a numeric vector of density values.
- `pmixnorm()` returns a numeric vector of probabilities.
- `qmixnorm()` returns a numeric vector of quantiles.
- `rmixnorm()` returns a tibble with simulated component values, component indicators, and the combined random draw.

Note

Version 1.0.1.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
mean <- c(-1, 1)
sd <- c(0.5, 1)
alpha <- c(0.4, 0.6)
dmixnorm(c(-1, 0, 1), mean, sd, alpha)
pmixnorm(c(-1, 0, 1), mean, sd, alpha)
qmixnorm(c(0.25, 0.75), mean, sd, alpha)

set.seed(1)
rmixnorm(3, mean, sd, alpha)
```

dsnrm

Skew-normal distribution

Description

Density, distribution function, quantile function, and random generation for the skew-normal distribution.

Usage

```
dsnrm(x, location = 0, scale = 1, shape = 0, log = FALSE)

psnrm(q, location = 0, scale = 1, shape = 0, log.p = FALSE, lower.tail = TRUE)

qsnrm(p, location = 0, scale = 1, shape = 0, log.p = FALSE, lower.tail = TRUE)

rsnrm(n, location = 0, scale = 1, shape = 0)
```

Arguments

<code>x</code>	Numeric vector of quantiles.
<code>location</code>	Numeric location parameter.
<code>scale</code>	Numeric scale parameter.
<code>shape</code>	Numeric shape parameter controlling skewness.
<code>log</code>	Logical. If TRUE, <code>dsnrm()</code> returns log-densities.
<code>q</code>	Numeric vector of quantiles.
<code>log.p</code>	Logical. If TRUE, probabilities are supplied or returned on the log scale.
<code>lower.tail</code>	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
<code>p</code>	Numeric vector of probabilities.
<code>n</code>	Number of observations.

Value

- `dsnrm()` returns a numeric vector of density values.
- `psnorm()` returns a numeric vector of probabilities.
- `qsnrm()` returns a numeric vector of quantiles.
- `rsnorm()` returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
dsnrm(c(-1, 0, 1), shape = 2)
psnorm(c(-1, 0, 1), shape = 2)
qsnrm(c(0.25, 0.75), shape = 2)

set.seed(1)
rsnorm(3, shape = 2)
```

dsolarGHI

Solar radiation distribution

Description

Density, distribution function, quantile function, and random generation for transformed global horizontal irradiance (GHI).

Usage

```
dsolarGHI(x, Ct, alpha, beta, pdf_Y, log = FALSE, link = "invgumbel")
```

```
psolarGHI(
  x,
  Ct,
  alpha,
  beta,
  cdf_Y,
  log.p = FALSE,
  lower.tail = TRUE,
  link = "invgumbel"
)
```

```
qsolarGHI(
  p,
  Ct,
  alpha,
  beta,
  cdf_Y,
  log.p = FALSE,
  lower.tail = TRUE,
  link = "invgumbel"
)
```

```
rsolarGHI(n, Ct, alpha, beta, cdf_Y, link = "invgumbel")
```

Arguments

x	Numeric vector of quantiles.
Ct	Numeric scalar or vector of clear-sky radiation values.
alpha	Numeric scalar. Lower transformation parameter.
beta	Numeric scalar. Scale transformation parameter. Typically $\beta > 0$ and $\alpha + \beta < 1$.
pdf_Y	Function. Density function of the latent variable Y .
log	Logical. If TRUE, dsolarGHI() returns log-densities.
link	Character string specifying the transformation link. Supported values are "invgumbel", "gumbel", "logis", and "norm".
cdf_Y	Function. Distribution function of the latent variable Y .
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
p	Numeric vector of probabilities.

Details

Consider a latent random variable Y with density pdf_Y and distribution function cdf_Y. With the inverse Gumbel link, the transformed solar radiation variable is

$$R_t(y) = C(t)(1 - \alpha - \beta \exp(-\exp(y)))$$

with support $[C(t)(1 - \alpha - \beta), C(t)(1 - \alpha)]$.

Value

- `dsolarGHI()` returns a numeric vector of density values.
- `psolarGHI()` returns a numeric vector of probabilities.
- `qsolarGHI()` returns a numeric vector of quantiles.
- `rsolarGHI()` returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
alpha <- 0.001
beta <- 0.9
Ct <- 7
dsolarGHI(c(3, 5), Ct, alpha, beta, dnorm)
psolarGHI(c(3, 5), Ct, alpha, beta, pnorm)
qsolarGHI(c(0.1, 0.9), Ct, alpha, beta, pnorm)

set.seed(1)
rsolarGHI(3, Ct, alpha, beta, pnorm)
```

dsolarK

Clearness index distribution

Description

Density, distribution function, quantile function, and random generation for the clearness index.

Usage

```
dsolarK(x, alpha, beta, pdf_Y, log = FALSE)

psolarK(x, alpha, beta, cdf_Y, log.p = FALSE, lower.tail = TRUE)

qsolarK(p, alpha, beta, cdf_Y, log.p = FALSE, lower.tail = TRUE)

rsolarK(n, alpha, beta, cdf_Y)
```

Arguments

<code>x</code>	Numeric vector of quantiles.
<code>alpha</code>	Numeric scalar. Lower transformation parameter.
<code>beta</code>	Numeric scalar. Scale transformation parameter. Typically $\beta > 0$ and $\alpha + \beta < 1$.

pdf_Y	Function. Density function of the latent variable Y .
log	Logical. If TRUE, dsolarK() returns log-densities.
cdf_Y	Function. Distribution function of the latent variable Y .
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
p	Numeric vector of probabilities.

Details

Consider a latent random variable Y with density pdf_Y and distribution function cdf_Y. The clearness index is modeled as

$$K(Y) = 1 - \alpha - \beta \exp(-\exp(Y))$$

with support $[1 - \alpha - \beta, 1 - \alpha]$.

Value

- dsolarK() returns a numeric vector of density values.
- psolarK() returns a numeric vector of probabilities.
- qsolarK() returns a numeric vector of quantiles.
- rsolarK() returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
alpha <- 0.001
beta <- 0.9
dsolarK(c(0.2, 0.5), alpha, beta, dnorm)
psolarK(c(0.2, 0.5), alpha, beta, pnorm)
qsolarK(c(0.1, 0.9), alpha, beta, pnorm)

set.seed(1)
rsolarK(3, alpha, beta, pnorm)
```

dsolarX	<i>Solar risk driver distribution</i>
---------	---------------------------------------

Description

Density, distribution function, quantile function, and random generation for the solar risk driver.

Usage

```
dsolarX(x, alpha, beta, pdf_Y, log = FALSE)

psolarX(x, alpha, beta, cdf_Y, log.p = FALSE, lower.tail = TRUE)

qsolarX(p, alpha, beta, cdf_Y, log.p = FALSE, lower.tail = TRUE)

rsolarX(n, alpha, beta, cdf_Y)
```

Arguments

x	Numeric vector of quantiles.
alpha	Numeric scalar. Lower transformation parameter.
beta	Numeric scalar. Scale transformation parameter. Typically $\beta > 0$ and $\alpha + \beta < 1$.
pdf_Y	Function. Density function of the latent variable Y .
log	Logical. If TRUE, dsolarX() returns log-densities.
cdf_Y	Function. Distribution function of the latent variable Y .
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
p	Numeric vector of probabilities.

Details

Consider a latent random variable Y with density pdf_Y and distribution function cdf_Y. The solar risk driver is modeled as

$$X(Y) = \alpha + \beta \exp(-\exp(Y))$$

with support $[\alpha, \alpha + \beta]$.

Value

- dsolarX() returns a numeric vector of density values.
- psolarX() returns a numeric vector of probabilities.
- qsolarX() returns a numeric vector of quantiles.
- rsolarX() returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsugeno\(\)](#), [dtnorm\(\)](#)

Examples

```
alpha <- 0.001
beta <- 0.9
dsolarX(c(0.2, 0.5), alpha, beta, dnorm)
psolarX(c(0.2, 0.5), alpha, beta, pnorm)
qsolarX(c(0.1, 0.9), alpha, beta, pnorm)

set.seed(1)
rsolarX(3, alpha, beta, pnorm)
```

dsugeno

Sugeno-distorted density and distribution

Description

Compute the Sugeno-distorted distribution for a given distribution or density.

Usage

```
dsugeno(pdf, cdf, lambda = 0)

psugeno(cdf, lambda = 0)
```

Arguments

pdf	Function, density function.
cdf	Function, distribution function.
lambda	Numeric, distortion parameter.

Value

- `dsugeno()` returns a density function.
- `psugeno()` returns a distribution function.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dtnorm\(\)](#)

Examples

```

cdf <- function(x) pnorm(x)
pdf <- function(x) dnorm(x)
x <- c(-1, 0, 1)
psugeno(cdf, lambda = -0.2)(x)
dsugeno(pdf, cdf, lambda = -0.2)(x)

```

dtnorm	<i>Truncated normal distribution</i>
--------	--------------------------------------

Description

Density, distribution function, quantile function, and random generation for a normal distribution truncated to an interval.

Usage

```

dtnorm(x, mean = 0, sd = 1, a = -3, b = 3, log = FALSE)

ptnorm(x, mean = 0, sd = 1, a = -3, b = 3, log.p = FALSE, lower.tail = TRUE)

qtnorm(p, mean = 0, sd = 1, a = -3, b = 3, log.p = FALSE, lower.tail = TRUE)

rtnorm(n, mean = 0, sd = 1, a = -100, b = 100)

```

Arguments

x	Numeric vector of quantiles.
mean	Numeric mean parameter.
sd	Numeric standard deviation parameter.
a	Numeric lower truncation bound.
b	Numeric upper truncation bound.
log	Logical. If TRUE, dtnorm() returns log-densities.
log.p	Logical. If TRUE, probabilities are supplied or returned on the log scale.
lower.tail	Logical. If TRUE, probabilities are $P[X \leq x]$; otherwise, $P[X > x]$.
p	Numeric vector of probabilities.
n	Number of observations.

Value

- dtnorm() returns a numeric vector of density values.
- ptnorm() returns a numeric vector of probabilities.
- qtnorm() returns a numeric vector of quantiles.
- rtnorm() returns a numeric vector of random draws.

Note

Version 1.0.0.

See Also

Other distributions: [desscher\(\)](#), [desscherMixture\(\)](#), [dgumbel\(\)](#), [dinvgumbel\(\)](#), [dkumaraswamy\(\)](#), [dmixnorm\(\)](#), [dsnrm\(\)](#), [dsolarGHI\(\)](#), [dsolarK\(\)](#), [dsolarX\(\)](#), [dsugeno\(\)](#)

Examples

```
dtnorm(c(-1, 0, 1), mean = 0, sd = 1, a = -2, b = 2)
ptnorm(c(-1, 0, 1), mean = 0, sd = 1, a = -2, b = 2)
qtnorm(c(0.25, 0.75), mean = 0, sd = 1, a = -2, b = 2)

set.seed(1)
rtnorm(3, mean = 0, sd = 1, a = -2, b = 2)
```

GARCH_companion_matrix

Construct the companion matrix of an GARCH model

Description

Construct the companion matrix of an GARCH model

Usage

```
GARCH_companion_matrix(alpha = 0, beta = 0)
```

Arguments

phi	numeric vector of length p, AR parameters.
theta	numeric vector of length q, MA parameters.

Value

numeric matrix of size p+q x p+q.

Note

Version 1.0.3

Examples

```
# ARCH(1) / GARCH(1) ~ Base 2 x 2
GARCH_companion_matrix(alpha = c(0.4))
GARCH_companion_matrix(beta = c(0.4))
# Only ARCH + 1 column of zero
GARCH_companion_matrix(alpha = c(0.4, 0.3, 0.1))
# Only GARCH + 1 column of zero
GARCH_companion_matrix(beta = c(0.4, 0.3, 0.1))
# FULL
GARCH_companion_matrix(c(0.4, 0.2), c(0.3, 0))
```

gaussianMixture	<i>Gaussian mixture model</i>
-----------------	-------------------------------

Description

Gaussian mixture model

Gaussian mixture model

Details

R6 class for fitting and storing a univariate Gaussian mixture model.

Applied after updating the parameters

Value

An R6 object of class gaussianMixture.

Public fields

control List to contain custom control parameters.

maxit Integer, maximum number of iterations.

maxrestarts Integer, maximum number of restarts.

abstol Numeric, absolute level for convergence.

components Integer, number of mixture components.

method Character, package used for fitting.

Active bindings

means Numeric vector containing the location parameter for each component.

sd Numeric vector containing the scale parameter for each component.

p Numeric vector containing the probability for each component.

coefficients named list with mixture coefficients.

std.errors named list with mixture parameters.

model Tibble with mixture parameters, in order means, sd, p.

loglik log-likelihood of the fitted series.

fitted fitted series

responsabilities resp

moments Tibble with the theoric moments and the number of observations used for fit.

tidy Tibble with estimated parameters, std.errors and statistics

Methods

Public methods:

- `gaussianMixture$new()`
- `gaussianMixture$set_time_series()`
- `gaussianMixture$logLik()`
- `gaussianMixture$E_step()`
- `gaussianMixture$classify()`
- `gaussianMixture$fit()`
- `gaussianMixture$EM()`
- `gaussianMixture$update()`
- `gaussianMixture$update_logLik()`
- `gaussianMixture$filter()`
- `gaussianMixture$Hessian()`
- `gaussianMixture$print()`
- `gaussianMixture$clone()`

Method `new()`: Initialize a gaussianMixture object.

Usage:

```
gaussianMixture$new(
  components = 2,
  method = "mixtools",
  maxit = 5000,
  maxrestarts = 500,
  abstol = 1e-08
)
```

Arguments:

`components` (`integer(1)`), number of components.

`method` Character, package used to fit the parameters. Can be `mclust` or `mixtools`.

`maxit` (`integer(1)`) Numeric, maximum number of iterations.

`maxrestarts` (`integer(1)`) Numeric, maximum number of restarts.

`abstol` (`numeric(1)`) Numeric, absolute level for convergence.

Method `set_time_series()`: Set the time series internally

Usage:

```
gaussianMixture$set_time_series(x, weights)
```

Arguments:

`x` vector

`weights` observations weights, if a weight is equal to zero the observation is excluded, otherwise is included with unitary weight.

Method `logLik()`: Compute the log-likelihood.

Usage:

```
gaussianMixture$logLik(x, params, per_obs = FALSE)
```

Arguments:

`x` vector

`params` Optional. Named list with mixture parameters.

per_obs Logical, when TRUE the log-likelihood is returned per observation, otherwise is summed.

Method E_step(): Compute the posterior probabilities (E-step),

Usage:

```
gaussianMixture$E_step(x, params)
```

Arguments:

x Time series to fit.

params A named list with mixture parameters.

Method classify(): Classify the time series in the components with highest likelihood.

Usage:

```
gaussianMixture$classify(x)
```

Arguments:

x Time series to fit.

Method fit(): Fit the parameters with mclust package

Usage:

```
gaussianMixture$fit(x, weights, mu_target = NA, var_target = NA)
```

Arguments:

x vector

weights observations weights, if a weight is equal to zero the observation is excluded, otherwise is included with unitary weight.

mu_target Numeric, target mean of the mixture to match.

var_target Numeric, target variance of the mixture to match. When missing all the available observations will be used.

Method EM(): Fit the parameters (EM-algorithm)

Usage:

```
gaussianMixture$EM(x, weights)
```

Arguments:

x vector

weights observations weights, if a weight is equal to zero the observation is excluded, otherwise is included with unitary weight. When missing all the available observations will be used.

Method update(): Update the parameters inside the object.

Usage:

```
gaussianMixture$update(means, sd, p)
```

Arguments:

means Named vector, new means parameters.

sd Named vector, new std. deviation parameters.

p Named vector, new probabilities.

Method update_logLik(): Update the log-likelihood with the current parameters

Usage:

```
gaussianMixture$update_logLik()
```

Method filter(): Update the responsibilities, the log-likelihood, classify again the points and recompute empiric parameters.

Usage:

```
gaussianMixture$filter()
```

Method Hessian(): Hessian matrix gaussianMixture class.

Usage:

```
gaussianMixture$Hessian()
```

Method print(): Print method for gaussianMixture class.

Usage:

```
gaussianMixture$print(label)
```

Arguments:

label Character, optional label.

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
gaussianMixture$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.2

Examples

```
gm <- gaussianMixture$new(components = 2)
gm$update(
  means = c(mu1 = -1, mu2 = 1),
  sd = c(sd1 = 0.5, sd2 = 1),
  p = c(p1 = 0.4, p2 = 0.6)
)
gm$coefficients
gm$moments
```

is_leap_year

Is leap year?

Description

Check if a given year is leap (366 days) or not (365 days).

Usage

```
is_leap_year(x)
```

Arguments

x numeric value or dates vector in the format YYYY-MM-DD.

Value

Boolean. TRUE if it is a leap year, FALSE otherwise.

Examples

```
is_leap_year("2024-02-01")
is_leap_year(c(2023:2030))
is_leap_year(c("2024-10-01", "2025-10-01"))
is_leap_year("2029-02-01")
```

ks_test	<i>Kolmogorov Smirnov test for a distribution</i>
---------	---

Description

Test against a specific distribution

Usage

```
ks_test(  
  x,  
  cdf,  
  ci = 0.05,  
  min_quantile = 0.015,  
  max_quantile = 0.985,  
  k = 1000,  
  plot = FALSE  
)
```

Arguments

x	a vector.
cdf	a function. The theoretic distribution to use for comparison.
ci	p.value for rejection.
min_quantile	minimum quantile for the grid of values.
max_quantile	maximum quantile for the grid of values.
k	finite value for approximation of infinite sum.
plot	when TRUE a plot is returned, otherwise a tibble.
seed	random seed for two sample test.

Value

when plot = TRUE a plot is returned, otherwise a tibble.

ks_test_ts	<i>Two sample Kolmogorov Smirnov test for a time series</i>
------------	---

Description

Perform a two sample invariance test for a time series.

Usage

```
ks_test_ts(
  x,
  ci = 0.05,
  idx_split,
  min_quantile = 0.015,
  max_quantile = 0.985,
  seed = 1,
  plot = FALSE
)
```

Arguments

x	a vector.
ci	p.value for rejection.
idx_split	Index used for splitting the time series. If missing will be random sampled.
min_quantile	minimum quantile for the grid of values.
max_quantile	maximum quantile for the grid of values.
seed	random seed for two sample test.
plot	when TRUE a plot is returned, otherwise a tibble.

logLik_std.errors	<i>Compute the standard errors and robust standard errors</i>
-------------------	---

Description

Compute the standard errors and robust standard errors

Usage

```
logLik_std.errors(params, logLik, robust = TRUE)
```

mix2norm_M_Yl_given_Yv

Conditional means component l given $Y_v = y_v$

Description

Conditional means component l given $Y_v = y_v$

Usage

```
mix2norm_M_Yl_given_Yv(y_v, M_Yl, M_Yv, S_Yv, Cv_Yl_Yv)
```

Arguments

y_v	conditioning value $Y_v = y_v$
M_Yl	mean vector for component l (state 1, state0)
M_Yv	mean vector for component v (state 1, state0)
S_Yv	std. deviation vector for component v (state 1, state0)
Cv_Yl_Yv	covariance between Yl and Yv.

mix2norm_probs_given_Yv

Conditional probabilities component l given $Y_v = y_v$

Description

Conditional probabilities component l given $Y_v = y_v$

Usage

```
mix2norm_probs_given_Yv(y_v, M_Yv, S_Yv, probs)
```

Arguments

y_v	conditioning value $Y_v = y_v$
M_Yv	mean vector for component v (state 1, state0)
S_Yv	std. deviation vector for component v (state 1, state0)
probs	Joint probabilities

```
mix2norm_S_Yl_given_Yv
```

Conditional std. deviation component l given $Y_v = y_v$

Description

Conditional std. deviation component l given $Y_v = y_v$

Usage

```
mix2norm_S_Yl_given_Yv(S_Yl, S_Yv, Cv_Yl_Yv)
```

Arguments

S_Yl	std. deviation vector for component l (state 1, state0)
S_Yv	std. deviation vector for component v (state 1, state0)
Cv_Yl_Yv	covariance between Yl and Yv.

```
monthlyParams
```

Create a function of time for monthly parameters

Description

Create a function of time for monthly parameters

Create a function of time for monthly parameters

Active bindings

parameters vector of parameters with length 12.

Methods

Public methods:

- `monthlyParams$new()`
- `monthlyParams$predict()`
- `monthlyParams$update()`
- `monthlyParams$clone()`

Method `new()`: Initialize a monthlyParams object

Usage:

```
monthlyParams$new(params)
```

Arguments:

params numeric vector of parameters with length 12.

Method `predict()`: Predict the monthly paramete

Usage:

```
monthlyParams$predict(x)
```

Arguments:
x date as character or month as numeric.

Method update(): Update the monthly parameters

Usage:
monthlyParams\$update(params)
Arguments:
params numeric vector of parameters with length 12.

Method clone(): The objects of this class are cloneable with this method.

Usage:
monthlyParams\$clone(deep = FALSE)
Arguments:
deep Whether to make a deep clone.

Note

Version 1.0.0

Examples

```
set.seed(1)
params <- runif(12)
mp <- monthlyParams$new(params)
t_now <- as.Date("2022-01-01")
t_hor <- as.Date("2024-12-31")
dates <- seq.Date(t_now, t_hor, by = "1 day")
plot(mp$predict(dates), type = "l")
```

number_of_day	<i>Number of day</i>
---------------	----------------------

Description

Compute the number of day of the year given a vector of dates.

Usage

```
number_of_day(x)
```

Arguments

x dates vector in the format YYYY-MM-DD.

Value

Numeric vector with the number of the day during the year.

Examples

```
number_of_day("2040-01-31")
number_of_day(c("2015-03-31", "2016-03-31", "2017-03-31"))
number_of_day(c("2015-02-28", "2016-02-28", "2017-02-28"))
number_of_day(c("2015-03-01", "2016-03-01", "2017-03-01"))
```

PDF

*Construct density, distribution, and quantile functions***Description**

[Experimental] Return functions with density-like, distribution-like, or quantile-like interfaces from a supplied density or distribution function.

Usage

```
PDF(.f, ...)

CDF(.f, lower = -Inf, ...)

Quantile(cdf, interval = c(-100, 100))
```

Arguments

<code>.f</code>	Function. Density function used by <code>PDF()</code> or integrated by <code>CDF()</code> .
<code>...</code>	Additional arguments passed to <code>.f</code> .
<code>lower</code>	Numeric. Lower integration bound used by <code>CDF()</code> .
<code>cdf</code>	Function. Cumulative distribution function used by <code>Quantile()</code> .
<code>interval</code>	Numeric vector of length two. Search interval used by <code>Quantile()</code> when solving for roots.

Value

- `PDF()` returns a function with arguments `x` and `log`.
- `CDF()` returns a function with arguments `x`, `lower.tail`, and `log.p`.
- `Quantile()` returns a function with arguments `p`, `log.p`, and `lower.tail`.

Examples

```
pdf <- PDF(dnorm, mean = 0.3, sd = 1.3)
pdf(3)

cdf <- CDF(dnorm, mean = 0.3, sd = 1.3)
cdf(3)

pnorm(Quantile(pnorm)(0.9))
```

PIT	<i>PIT computation</i>
-----	------------------------

Description

PIT computation

Usage

```
PIT_psolarGHI(Rt, Ct, alpha, beta, M_Y, S_Y, probs, link = "invgumbel")
```

Arguments

Rt vector of solar radiation.
 Ct vector of clear-sky radiation.

PIT_test	<i>Perform KS and LB test on PIT grades</i>
----------	---

Description

Perform KS and LB test on PIT grades

Usage

```
PIT_test(u, type = "bp", lag = 5)
```

Arguments

u vector, PIT.
 type type of autocorrelation test ("bp" or "lb")
 lag vector of lags at which perform autocorrelation test.

riccati_root	<i>Riccati Root</i>
--------------	---------------------

Description

Compute the square root of a symmetric matrix.

Usage

```
riccati_root(x)
```

Arguments

x squared and symmetric matrix.

Examples

```
cv <- matrix(c(1, 0.3, 0.3, 1), nrow = 2, byrow = TRUE)
riccati_root(cv)
```

seasonalClearsky

Seasonal clear-sky model

Description

R6 implementation of a clear-sky seasonal model for solar radiation, using extraterrestrial radiation and seasonal harmonics.

Super class

[solarr::seasonalModel](#) -> seasonalClearsky

Public fields

lat Numeric, scalar, latitude of the location considered.
spec List, model specification

Active bindings

ssf See [seasonalSolarFunctions](#) for more details.

Methods**Public methods:**

- [seasonalClearsky\\$new\(\)](#)
- [seasonalClearsky\\$fit\(\)](#)
- [seasonalClearsky\\$predict\(\)](#)
- [seasonalClearsky\\$differential\(\)](#)
- [seasonalClearsky\\$print\(\)](#)
- [seasonalClearsky\\$clone\(\)](#)

Method [new\(\)](#): Initialize a seasonalClearsky object.

Usage:

```
seasonalClearsky$new(
  spec = seasonalClearsky_spec(),
  control = control_seasonalClearsky()
)
```

Arguments:

spec Named list, model's specification. See the function [seasonalClearsky_spec](#) for more details.

control Named list, control parameters. See the function [control_seasonalClearsky](#) for more details.

Method [fit\(\)](#): Fit the seasonal model for clear sky radiation.

Usage:

```
seasonalClearsky$fit(x, dates, lat, clearsky)
```

Arguments:

x Numeric vector, time series of solar radiation.

dates Character or Date vector, time series of dates.

lat Numeric scalar, reference latitude.

clearsky Numeric vector, time series of target clear sky radiation.

Method `predict()`: Predict method for seasonalClearsky object.

Usage:

```
seasonalClearsky$predict(n, newdata)
```

Arguments:

n Integer, scalar or vector. number of day of the year.

newdata An optional data frame in which to look for variables with which to predict. If omitted, the fitted values are used.

Method `differential()`: Differential method for seasonalClearsky object.

Usage:

```
seasonalClearsky$differential(n, newdata)
```

Arguments:

n Integer, scalar or vector. number of day of the year.

newdata An optional data frame in which to look for variables with which to predict. If omitted, the fitted values are used.

Method `print()`: Print method for seasonalClearsky object.

Usage:

```
seasonalClearsky$print()
```

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
seasonalClearsky$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.3

seasonalClearsky_spec *Specification for seasonalClearsky model*

Description

Specification for seasonalClearsky model

Usage

```
seasonalClearsky_spec(
  method = "OLS",
  order = 1,
  order_H0 = 1,
  method_H0 = "spencer",
  period = 365,
  include.intercept = TRUE,
  include.trend = FALSE
)
```

Arguments

method	Character, method of fitting: can be: OLS, RLS, CLS
order	Integer scalar, number of combinations of sines and cosines.
order_H0	Integer scalar, order of the polynomial for extra-terrestrial radiation.
method_H0	Character, method of computation of the extra-terrestrial radiation: can be: spencer, cooper.
period	Integer scalar, seasonality period. The default is 365.
include.intercept	Logical, when TRUE, the default, the intercept will be included in the clear sky model.

Value

Named list of control parameters.

Note

Version 1.0.0

Examples

```
spec_Ct = spec_seasonalClearsky()
```

seasonalModel

*Seasonal Model***Description**

The `seasonalModel` class implements a seasonal regression model as a linear combination of sine and cosine functions. This model is designed to capture periodic effects in time series data, particularly for applications involving seasonal trends.

Details

The seasonal model is fitted using a specified formula, which allows for the inclusion of external regressors along with sine and cosine terms to model seasonal variations. The periodicity can be customized, and the model can be updated with new coefficients after the initial fit.

Public fields

`extra_params` List to contain custom extra parameters.

`control` List to contain custom control parameters.

Active bindings

`period` Integer vector, periodicities of the seasonality.

`order` Integer vector, number of combinations of sines and cosines.

`omega` Integer, periodicity computed as $\frac{2\pi}{\text{period}}$.

`coefficients` Named vector, model's parameters.

`dcoefficients` Named vector, model's parameters for the differential with respect to `n`.

`std.errors` Named vector, standard errors of the model's parameters.

`coefs_names` Named vector, names of the model's parameters.

`tidy` A tibble with estimated parameters and std. errors.

Methods**Public methods:**

- `seasonalModel$new()`
- `seasonalModel$fit()`
- `seasonalModel$predict()`
- `seasonalModel$differential()`
- `seasonalModel$update()`
- `seasonalModel$update_coefs_names()`
- `seasonalModel$update_std.errors()`
- `seasonalModel$print()`
- `seasonalModel$clone()`

Method `new()`: Initialize a `seasonalModel` object.

Usage:

```
seasonalModel$new(formula, order = 1, period = 365)
```

Arguments:

formula *formula*, an object of class *formula* (or one that can be coerced to that class). It is a symbolic description of the model to be fitted and can be used to include or exclude the intercept or external regressors in data.

order Integer, number of combinations of sines and cosines.

period Integer, seasonality period. The default is 365.

Method *fit()*: Fit a seasonal model as a linear combination of sine and cosine functions and eventual external regressors specified in the formula. The external regressors used should have the same periodicity, i.e. not stochastic regressors are allowed.

Usage:

```
seasonalModel$fit(data, ...)
```

Arguments:

data A data frame, list or environment (or object coercible by *as.data.frame* to a data frame) containing the variables in the model.

... other parameters to be passed to the function *lm*.

Method *predict()*: Predict method for the class *seasonalModel*.

Usage:

```
seasonalModel$predict(n, newdata)
```

Arguments:

n Integer vector, numbers of day of the year.

newdata an optional data frame, list or environment (or object coercible by *as.data.frame* to a data frame) containing the variables in the model.

Method *differential()*: Compute the differential with respect to time (*n*).

Usage:

```
seasonalModel$differential(n, newdata)
```

Arguments:

n Integer, number of day of the year.

newdata an optional data frame, list or environment (or object coercible by *as.data.frame* to a data frame) containing the variables in the model.

Method *update()*: Update the parameters.

Usage:

```
seasonalModel$update(coefficients)
```

Arguments:

coefficients Named vector, with new parameters, the names should match the names of the coefficients.

Method *update_coefs_names()*: Update the names of the parameters.

Usage:

```
seasonalModel$update_coefs_names(coefs_names)
```

Arguments:

coefs_names Named vector, new parameters names, the names should match the names of the old coefficients.

Method `update_std.errors()`: Update the standard errors of the parameter.

Usage:

```
seasonalModel$update_std.errors(std.errors)
```

Arguments:

`std.errors` Named vector with new standard errors, the names should match the names of the coefficients.

Method `print()`: Print method for the class `seasonalModel`.

Usage:

```
seasonalModel$print()
```

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
seasonalModel$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Note

Version 1.0.3

Examples

```
# Sample time series
n <- 1000
eps <- rnorm(n, sd = 0.2)
Yt <- 0.5 + 0.2 * sin(2*pi/365*c(1:n)) + 0.3 * cos(2*pi/365*c(1:n)) + eps

# Initialize the model
sm <- seasonalModel$new("Yt ~ 1", 1, 365)
# Fit the model
data = data.frame(Yt = Yt, n = 1:1000)
sm$fit(data = data)
data$Yt_hat <- sm$predict(n = data$n)
# Fitted parameters
sm$coefficients

# Plot the fitted data
library(ggplot2)
ggplot()+
  geom_line(data = data, aes(n, Yt))+
  geom_line(data = data, aes(n, Yt_hat), color = "red")
```

seasonalSolarFunctions

Solar geometry and extraterrestrial radiation utilities

Description

seasonalSolarFunctions is an R6 class that implements standard solar geometry and clear-sky formulas (Duffie & Beckman, Comini).

Details

Implementations includes:

- Day-of-year seasonal parameter $B(n)$ and time equation $E(n)$
- Angle conversions between radians and degrees
- Solar time, hour angle, declination, and sunset hour angle
- Extraterrestrial radiation on a horizontal surface H_{on}
- Solar incidence, azimuth, altitude, and daily solar angles
- Hottel-type clear-sky transmissivity and clear-sky irradiance

The methods follow the notation and equations in Duffie & Beckman (*Solar Engineering of Thermal Processes*, 4th ed.) unless otherwise specified.

Public fields

legal_hour Logical, when TRUE the clock time will be corrected for the legal hour.

Active bindings

Gsc solar constant, i.e, 1367.

Methods

Public methods:

- `seasonalSolarFunctions$new()`
- `seasonalSolarFunctions$update_method()`
- `seasonalSolarFunctions$B()`
- `seasonalSolarFunctions$degree()`
- `seasonalSolarFunctions$radiant()`
- `seasonalSolarFunctions$E()`
- `seasonalSolarFunctions$elevation()`
- `seasonalSolarFunctions$solar_time()`
- `seasonalSolarFunctions$solar_hour()`
- `seasonalSolarFunctions$hour_angle()`
- `seasonalSolarFunctions$incidence_angle()`
- `seasonalSolarFunctions$azimut_angle()`
- `seasonalSolarFunctions$Gon()`
- `seasonalSolarFunctions$declination()`

- `seasonalSolarFunctions$Hon()`
- `seasonalSolarFunctions$sunset_hour_angle()`
- `seasonalSolarFunctions$sun_hours()`
- `seasonalSolarFunctions$solar_altitude()`
- `seasonalSolarFunctions$solar_angles()`
- `seasonalSolarFunctions$clearsky()`
- `seasonalSolarFunctions$clone()`

Method `new()`: Initialize a `seasonalSolarFunctions` object

Usage:

```
seasonalSolarFunctions$new(method = "spencer", legal_hour = TRUE)
```

Arguments:

`method` character, method type for computations. Can be `cooper` or `spencer`.

`legal_hour` Logical, when TRUE the clock time will be corrected for the legal hour.

Method `update_method()`: Extract or update the method used for computations.

Usage:

```
seasonalSolarFunctions$update_method(x)
```

Arguments:

`x` character, method type. Can be `cooper` or `spencer`.

Returns: When `x` is missing it return a character containing the method that is actually used.

Method `B()`: Seasonal adjustment parameter.

Usage:

```
seasonalSolarFunctions$B(n)
```

Arguments:

`n` number of the day of the year

Details: The function implement Eq. 1.4.2 from Duffie (4th edition), i.e.

$$B(n) = \frac{2\pi}{365}n$$

Method `degree()`: Convert angles in radian into an angles in degrees.

Usage:

```
seasonalSolarFunctions$degree(x)
```

Arguments:

`x` numeric vector, angles in radian.

Details: The function computes:

$$\frac{x180}{\pi}$$

Method `radiant()`: Convert angles in degrees into an angles in radian

Usage:

```
seasonalSolarFunctions$radiant(x)
```

Arguments:

`x` numeric vector, angles in degrees.

Details: The function computes:

$$\frac{x\pi}{180}$$

Method E(): Compute the time adjustment in minutes.

Usage:

seasonalSolarFunctions\$E(n)

Arguments:

n number of the day of the year

Details: The function implement Eq. 1.5.3 from Duffie (4th edition), i.e.

$$E = 229.2(0.000075 + 0.001868 \cos(B) - 0.032077 \sin(B) - 0.014615 \cos(2B) - 0.04089 \sin(2B))$$

Returns: The time adjustment in minutes.

Method elevation(): Compute the angle in the degree given a certain altitude in meters.

Usage:

seasonalSolarFunctions\$elevation(alt)

Arguments:

alt Numeric, altitude in meters.

Method solar_time(): Compute the solar time from a clock time.

Usage:

seasonalSolarFunctions\$solar_time(x, lon, lon_st = 15, tz = "Europe/Rome")

Arguments:

x datetime, clock hour.

lon longitude of interest in degrees.

lon_st longitude of the Local standard meridian in degrees.

tz Character, reference time zone.

Details: The function implement Eq. 1.5.2 from Duffie (4th edition), i.e.

$$solar_{time} = clock_{time} + 4(lon - lon_{st}) + E(n)$$

Returns: A datetime object

Method solar_hour(): Compute the solar hour for a specific clock time.

Usage:

seasonalSolarFunctions\$solar_hour(LST)

Arguments:

LST datetime, true solar time.

Returns: Hours

Method hour_angle(): Compute the solar angle for a specific hour of the day.

Usage:

seasonalSolarFunctions\$hour_angle(LST)

Arguments:

LST datetime, true solar time.

Details: The function implement Eq. 1.42 from Comini (2013), i.e.

$$\omega = 15(solarhour - 12)$$

where the "solarhour" is expressed in hours.

Returns: An angle in degrees

Method incidence_angle(): Compute the incidence angle

Usage:

seasonalSolarFunctions\$incidence_angle(LST, lat, alt = 0, beta = 0, gamma = 0)

Arguments:

LST datetime, true solar time.

lat latitude of interest in degrees.

alt Numeric, altitude in meters.

beta altitude

gamma orientation

Returns: An angle in degrees

Method azimuth_angle(): Compute the solar azimuth angle for a specific time of the day.

Usage:

seasonalSolarFunctions\$azimut_angle(LST, lat, alt, beta = 0, gamma = 0)

Arguments:

LST datetime, true solar time.

lat latitude of interest in degrees.

alt Numeric, altitude in meters.

beta altitude

gamma orientation

Details: The function implement Eq. 1.6.6 from Duffie (4th edition), i.e.

$$\gamma_s = sign(\omega) \left| \cos^{-1} \left(\frac{\cos \theta_z \sin \phi - \sin \delta}{\sin \theta_z \cos \phi} \right) \right|$$

Returns: The solar azimuth angle in degrees

Method Gon(): Compute the solar constant adjusted for the day of the year.

Usage:

seasonalSolarFunctions\$Gon(n, deriv = FALSE)

Arguments:

n number of the day of the year.

deriv Logical, when TRUE will return the first derivative with respect to time.

Details: When method is cooper the function implement Eq. 1.4.1a from Duffie (4th edition), i.e.

$$G_{o,n} = G_{sc}(1 + 0.033 \cos(B))$$

otherwise when it is spencer it implement Eq. 1.4.1b from Duffie (4th edition):

$$G_{o,n} = G_{sc}(1.000110 + 0.034221 \cos(B) + 0.001280 \sin(B) + 0.000719 \cos(2B) + 0.000077 \sin(2B))$$

When `deriv = TRUE` it will be returned the derivatives with respect to time. When the method is cooper:

$$\frac{\partial G_{o,n}}{\partial n} = -G_{sc} \frac{2\pi}{365} 0.033 \sin(B)$$

Otherwise if it is spencer:

$$\frac{\partial G_{o,n}}{\partial n} = G_{sc} \frac{2\pi}{365} (-0.034221 \sin(B) + 0.001280 \cos(B) - 0.001438 \sin(2B) + 0.000154 \cos(2B))$$

Returns: The solar constant in W/m^2 for the day `n`.

Method `declination()`: Compute solar declination in degrees.

Usage:

`seasonalSolarFunctions$declination(n, deriv = FALSE)`

Arguments:

`n` number of the day of the year

`deriv` Logical, when `TRUE` will return the first derivative with respect to time.

Details: When method is cooper the function implement Eq. 1.6.1a from Duffie (4th edition), i.e.

$$\delta(n) = 23.45 \sin\left(\frac{2\pi(284 + n)}{365}\right)$$

otherwise when it is spencer it implement Eq. 1.6.1b from Duffie (4th edition):

$$\delta(n) = \frac{180}{\pi} (0.006918 - 0.399912 \cos(B) + 0.070257 \sin(B) - 0.006758 \cos(2B) + 0.000907 \sin(2B) - 0.002697 \cos(3B) + 0.001481 \sin(3B) - 0.000190 \cos(4B) + 0.000075 \sin(4B))$$

When `deriv = TRUE` it will be returned the derivatives with respect to time. When the method is cooper:

$$\frac{\partial \delta}{\partial n}(n) = 23.45 \frac{2\pi}{365} \cos\left(\frac{2\pi(284 + n)}{365}\right)$$

otherwise when the method is spencer:

$$\frac{\partial \delta}{\partial n}(n) = \frac{360}{365} (0.399912 \sin(B) + 0.070257 \cos(B) + 0.013516 \sin(2B) + 0.001814 \cos(2B) + 0.008091 \sin(3B) + 0.000190 \cos(3B) - 0.000075 \sin(3B) - 0.000190 \cos(4B) - 0.000075 \sin(4B))$$

Returns: The solar declination in degrees.

Method `Hon()`: Compute the solar extraterrestrial radiation

Usage:

`seasonalSolarFunctions$Hon(n, lat, alt, deriv = FALSE)`

Arguments:

`n` number of the day of the year

`lat` latitude of interest in degrees.

`alt` Numeric, altitude in meters.

`deriv` Logical, when `TRUE` will return the first derivative with respect to time.

Details: The function implement Eq. 1.10.3 from Duffie (4th edition):

$$H_{on} = G_{on} \frac{24 \times 3600}{\pi} (\cos(lat) \cos(\delta) \sin(\omega_s) + \frac{\pi}{180} \sin(lat) \sin(\delta))$$

Returns: Extraterrestrial radiation on an horizontal surface in kilowatt hour for meters squared for day.

Method `sunset_hour_angle()`: Compute solar angle at sunset in degrees

Usage:

```
seasonalSolarFunctions$sunset_hour_angle(n, lat, alt, deriv = FALSE)
```

Arguments:

`n` number of the day of the year

`lat` Numeric, latitude of interest in degrees.

`alt` Numeric, altitude in meters.

`deriv` Logical, when TRUE will return the first derivative with respect to time.

Details: The function implement Eq. 1.6.10 from Duffie (4th edition), i.e.

$$\omega_s = \cos^{-1}(-\tan(\delta(n)) \tan(\phi))$$

When altitude is not missing it will implement a generalized version with altitude, i.e.

$$\omega_s = \cos^{-1} \left(\frac{\sin H - \sin \delta \sin \phi}{\cos \phi \cos \delta} \right)$$

Returns: The sunset hour angle in degrees.

Method `sun_hours()`: Compute number of sun hours for a day `n`.

Usage:

```
seasonalSolarFunctions$sun_hours(n, lat, alt)
```

Arguments:

`n` number of the day of the year.

`lat` Numeric, latitude of interest in degrees.

`alt` Numeric, altitude in meters.

Details: The function implement Eq. 1.6.11 from Duffie (4th edition), i.e.

$$\frac{2}{15} \omega_s$$

Method `solar_altitude()`: Compute solar altitude in degrees

Usage:

```
seasonalSolarFunctions$solar_altitude(n, lat)
```

Arguments:

`n` number of the day of the year

`lat` Numeric, latitude of interest in degrees.

Details: The function computes

$$\sin^{-1}(-\sin(\delta(n)) \sin(\phi) + \cos(\delta(n)) \cos(\phi))$$

Method `solar_angles()`: Compute the solar angle for a latitude in different dates.

Usage:

```
seasonalSolarFunctions$solar_angles(
  x,
  lat,
  lon,
  alt,
  lon_st = 15,
  beta = 0,
  gamma = 0,
  by = "1 min",
  tz = "Europe/Rome"
)
```

Arguments:

x datetime, clock hour.
lat Numeric, latitude of interest in degrees.
lon Numeric, longitude of interest in degrees.
alt Numeric, altitude in meters.
lon_st Numeric, longitude of the Local standard meridian in degrees
beta Numeric angle, inclination of the solar panel.
gamma Numeric, angle orientation of the panel.
by Character, time step. Default is 1 min.
tz Character, reference time zone.

Method clearsky(): Hottel clearsky*Usage:*

```
seasonalSolarFunctions$clearsky(
  cosZ = NULL,
  G0 = NULL,
  alt,
  clime = "No Correction"
)
```

Arguments:

cosZ solar incidence angle
G0 solar constant
alt Numeric, altitude in meters.
clime clime correction

Method clone(): The objects of this class are cloneable with this method.*Usage:*

```
seasonalSolarFunctions$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.2

Examples

```

dates <- seq.Date(as.Date("2022-01-01"), as.Date("2022-12-31"), 1)
# Seasonal functions object
sf <- seasonalSolarFunctions$new()

# Adjustment parameter
sf$B(number_of_day(dates))

# Time adjustment in minutes
sf$E(dates)

# Declination
sf$declination(dates)
```

```

# Solar constant
sf$Gsc

# Solar constant adjusted
sf$Gon(dates)

# Extraterrestrial radiation
sf$Hon(dates, 43)

# Number of hours of sun
sf$sun_hours(dates, 43)

# Sunset hour angle
sf$sunset_hour_angle(dates, 43)

sf$solar_time("2022-01-01 12:00", 11, 10)
sf$hour_angle("2022-01-01 14:00", 11, 15)
sf$incidence_angle("2022-06-01 21:00", 31, 12, lon_st = 15, beta = 0, gamma = 0)
sf$azimut_angle("2022-01-01 14:00", 30, 17, lon_st = 15)

```

sGARCH

*Implementation of rugarch methods for a GARCH(p,q) as R6 class***Description**

Implementation of rugarch methods for a GARCH(p,q) as R6 class

Implementation of rugarch methods for a GARCH(p,q) as R6 class

Public fields

control List to contain custom control parameters.

model_name Character, model name

Active bindings

archOrder Numeric scalar, ARCH order.

garchOrder Numeric scalar, GARCH order.

order Numeric named vector, orders of the GARCH model. The first element is the ARCH order, while the second the GARCH order.

omega Numeric scalar, intercept parameter.

alpha Numeric vector, ARCH parameters.

beta Numeric vector, GARCH parameters.

coefficients Numeric vector, model's coefficients.

A Numeric matrix, companion matrix.

b Numeric vector.

d Numeric vector

std.errors Numeric vector, standard errors of the model's coefficients.

sigma2_inf Numeric scalar, long-term unconditional std. deviation.

loglik model log-likelihood

tidy Tibble with estimated parameters and relative std. errors.

Methods

Public methods:

- `sGARCH$new()`
- `sGARCH$fit()`
- `sGARCH$filter()`
- `sGARCH$update()`
- `sGARCH$update_std.errors()`
- `sGARCH$next_step()`
- `sGARCH$print()`
- `sGARCH$clone()`

Method `new()`: Initialize a standard GARCH model

Usage:

```
sGARCH$new(archOrder = 1, garchOrder = 1, mode = "unitOmega")
```

Arguments:

`archOrder` Integer scalar, ARCH order.

`garchOrder` Integer scalar, GARCH order.

`mode` Character, one of "unitOmega", "targetSigma2", "freeOmega".

Method `fit()`: Fit the GARCH model with rugarch function.

Usage:

```
sGARCH$fit(x, weights)
```

Arguments:

`x` Numeric, vector. Time series to be fitted.

`weights` Numeric, vector. Optional custom weights.

Method `filter()`: Filter method from rugarch package to compute GARCH variance, residuals and log-likelihoods.

Usage:

```
sGARCH$filter(x, eps0 = NULL, sigma20 = NULL)
```

Arguments:

`x` Numeric, vector. Time series to be filtered.

`eps0` Optional numeric initial epsilons to prepend (length p+q).

`sigma20` Optional numeric initial variances to prepend (length p+q).

Method `update()`: Update the coefficients of the model

Usage:

```
sGARCH$update(coefficients)
```

Arguments:

`coefficients` Numeric, named vector. Model's coefficients.

Method `update_std.errors()`: Numerical computation of the std. errors of the parameters.

Usage:

```
sGARCH$update_std.errors(std.errors)
```

Arguments:

std.errors Numeric std. errors.

Method next_step(): Next step GARCH std. deviation forecast

Usage:

```
sGARCH$next_step(eps0, sigma20)
```

Arguments:

eps0 Numeric initial epsilons to prepend (length p+q).

sigma20 Numeric initial variances to prepend (length p+q).

Method print(): Print method for GARCH_modelR6 class.

Usage:

```
sGARCH$print()
```

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
sGARCH$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.3

solarHedging_model	<i>Global Minimum Variance number of contracts</i>
--------------------	--

Description

Compute the optimal number of contracts, such that the variance of the cash flow of a solar power producer is minimum.

Usage

```
solarHedging_model(
  model,
  moments,
  P0_P,
  r_star,
  gamma = 0.01,
  put = TRUE,
  control_options = control_solarOption(),
  control_hedge = control_solarHedging()
)
```

Arguments

model	An object with the class <code>solarModel</code> . See the function solarModel for more details.
moments	Tibble containing the forecasted moments for different days ahead. See the function solarMoments for more details.
P0_P	Optional numeric scalar, expected value of 1 solar derivative with unitary tick under $\mathbb{P}$.
gamma	Numeric scalar, risk aversion parameter.
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.
control_hedge	Named list, control parameters for hedging. See the function control_solarHedging for more details.

Note

Version 1.0.0.

solarHedging_scenarios

Compute the optimal number of solar derivative index

Description

Compute the optimal number of contracts, such that the variance of the cash flow of a solar power producer with a given setup is minimum.

Usage

```
solarHedging_scenarios(
  scenarios,
  P0_P,
  r_star = 0,
  gamma = 1e-04,
  put = TRUE,
  control_options = control_solarOption(),
  control_hedge = control_solarHedging()
)
```

Arguments

scenarios	Object with the class <code>solarScenario</code> . See the function solarScenario for more details.
P0_P	Optional numeric scalar, expected value of 1 solar derivative with unitary tick under $\mathbb{P}$.
gamma	Numeric scalar, risk aversion parameter.

put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.
control_hedge	Named list, control parameters for hedging. See the function control_solarHedging for more details.

Note

Version 1.0.0.

solarMixture	<i>Monthly Gaussian mixture with two components</i>
--------------	---

Description

Monthly Gaussian mixture with two components

Monthly Gaussian mixture with two components

Public fields

control List to contain custom control parameters.

mu1 Function, see [monthlyParams](#).

mu2 Function, see [monthlyParams](#).

sd1 Function, see [monthlyParams](#).

sd2 Function, see [monthlyParams](#).

prob Function, see [monthlyParams](#).

Active bindings

data A tibble with the following columns:

date Time series of dates.

Month Vector of Month.

x Time series used for fitting.

w Time series of weights.

means Matrix of means where a row represents a month and a column a mixture component.

sd Matrix of std. deviations where a row represents a month and a column a mixture component.

p Matrix of probabilities where a row represents a month and a column a mixture component.

model Named List with 12 [gaussianMixture](#) objects.

loglik Numeric, total log-likelihood.

fitted A tibble with the classified series

moments A tibble with the theoric moments. It contains:

Month Month of the year.

mean Theoric monthly expected value of the mixture model.

variance Theoric monthly variance of the mixture model.

skewness Theoric monthly skewness.

kurtosis Theoric monthly kurtosis.

nobs Number of observations used for fitting.

loglik Monthly log-likelihood.

coefficients A tibble with the fitted parameters.

std.errors A tibble with the fitted std.errors

tidy A tibble with the fitted std.errors

Methods

Public methods:

- `solarMixture$new()`
- `solarMixture$set_time_series()`
- `solarMixture$fit()`
- `solarMixture$update()`
- `solarMixture$update_logLik()`
- `solarMixture$filter()`
- `solarMixture$hessian()`
- `solarMixture$logLik()`
- `solarMixture$grades()`
- `solarMixture$VaR()`
- `solarMixture$ES()`
- `solarMixture$print()`
- `solarMixture$clone()`

Method `new()`: Initialize a `solarMixture` object

Usage:

```
solarMixture$new(
  components = 2,
  method = "mixtools",
  maxit = 10000,
  maxrestarts = 1000,
  abstol = 1e-08
)
```

Arguments:

components (integer(1)), number of components.

method Character, package used to fit the parameters. Can be `mixtools` or `mclust`.

maxit (integer(1)) Numeric, maximum number of iterations.

maxrestarts (integer(1)) Numeric, maximum number of restarts.

abstol (numeric(1)) Numeric, absolute level for convergence.

Method `set_time_series()`: Set the time series internally

Usage:

```
solarMixture$set_time_series(x, dates, weights)
```

Arguments:

x vector

dates date vector

weights observations weights, if a weight is equal to zero the observation is excluded, otherwise is included with unitary weight.

Method fit(): Fit the parameters with mclust package

Usage:

```
solarMixture$fit(
  x,
  date,
  weights,
  method = "mixtools",
  mu_target = rep(NA, 12),
  var_target = rep(NA, 12)
)
```

Arguments:

x vector

date date vector

weights observations weights, if a weight is equal to zero the observation is excluded, otherwise is included with unitary weight. When missing all the available observations will be used.

method Character, package used to fit the parameters. Can be mclust or mixtools.

mu_target Numeric vector with length 12, target mean of the mixture to match.

var_target Numeric vector with length 12, target variance of the mixture to match.

Method update(): Update means, sd, p .

Usage:

```
solarMixture$update(means, sd, p)
```

Arguments:

means Numeric matrix of means parameters.

sd Numeric matrix of std. deviation parameters.

p Numeric matrix of probability parameters.

Method update_logLik(): Apply the \$update_logLik() method to all the gaussianMixture models

Usage:

```
solarMixture$update_logLik()
```

Method filter(): Apply the \$filter() method to all the gaussianMixture models

Usage:

```
solarMixture$filter()
```

Method Hessian(): Apply the \$Hessian() method to all the gaussianMixture models

Usage:

```
solarMixture$Hessian()
```

Method logLik(): Log-likelihood

Usage:

```
solarMixture$logLik(x, date)
```

Arguments:

x vector
date dates

Method grades(): Compute the grades

Usage:

solarMixture\$grades(x, date)

Arguments:

x vector
date dates

Method VaR(): Compute the VaR with certain confidence levels

Usage:

solarMixture\$VaR(date, alpha = 0.05)

Arguments:

date dates
alpha confidence levels for the VaR
x vector

Method ES(): Compute the VaR with certain confidence levels

Usage:

solarMixture\$ES(date, alpha = 0.05)

Arguments:

date dates
alpha confidence levels for the VaR
x vector

Method print(): Print method for solarMixture class.

Usage:

solarMixture\$print()

Method clone(): The objects of this class are cloneable with this method.

Usage:

solarMixture\$clone(deep = FALSE)

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.0

Examples

```
# Model fit
model <- solarModel$new(spec)
model$fit()
# Inputs
x <- model$data$u_tilde
w <- model$data$weights
date <- model$data$date
```

```
# Solar Mixture object
sm <- solarMixture$new()
sm$fit(x, date, w)
params <- sm$parameters
sm$std.errors
# params[1,]$mu1 <- params[1,]$mu1*0.9
# sm$update(means = params[,c(2,3)])
```

solarMixture_VaR

*Compute the Value at Risk and Expected Shortfall of a SolarMixture***Description**

Compute the Value at Risk and Expected Shortfall of a SolarMixture

Usage

```
solarMixture_VaR(solarMix, date, x, alpha = 0.05, ci = 0.05, ES = FALSE)
```

Arguments

alpha	Numeric vector of confidence levels. Allows for more than one alpha.
ci	Numeric scalar, confidence levels used to state if the Null is rejected or not on VaR tests.
ES	Logical, when TRUE the expected shortfall will be also computed for each alpha.
model	solarMixture
type	Numeric, type of evaluation, full on the complete data, train on the train data, test on the test data.

solarModel

*Solar Model in R6 Class***Description**

The solarModel class combines seasonal components, bounded transformations, ARMA-GARCH dynamics, and monthly Gaussian mixture residual models for solar radiation data.

Details

The class stores a solarModel_spec object and fitted data summaries. Most public methods update the object state in place and are intended for the package fitting workflow.

Value

An R6 object of class solarModel.

Public fields

interpolated Logical values, when TRUE the time-series and the parameters are interpolated.

Active bindings

`data` A data frame with fitted data, seasonal quantities, and monthly parameters.

`seasonal_data` A data frame containing seasonal and monthly summaries.

`monthly_data` A data frame containing monthly parameters.

`loglik` The log-likelihood computed on train data.

`spec` A `solarModel_spec` object that controls the model fitting process.

`moments` Get a list containing the conditional moments.

`coefficients` Get the model parameters as a named list.

Methods**Public methods:**

- `solarModel$new()`
- `solarModel$fit()`
- `solarModel$fit_seasonal_model_Ct()`
- `solarModel$compute_risk_drivers()`
- `solarModel$fit_transform()`
- `solarModel$fit_seasonal_mean()`
- `solarModel$fit_monthly_mean()`
- `solarModel$fit_mean_model()`
- `solarModel$fit_seasonal_variance()`
- `solarModel$fit_monthly_variance()`
- `solarModel$correct_seasonal_variance()`
- `solarModel$fit_variance_model()`
- `solarModel$fit_mixture_model()`
- `solarModel$update()`
- `solarModel$update_moments()`
- `solarModel$update_logLik()`
- `solarModel$update_risk_drivers()`
- `solarModel$update_classification()`
- `solarModel$filter()`
- `solarModel$logLik()`
- `solarModel$print()`
- `solarModel$clone()`

Method `new()`: Initialize a `solarModel`

Usage:

```
solarModel$new(spec)
```

Arguments:

`spec` An object of class `solarModel_spec`. See [solarModel_spec](#).

Returns: Initializes object state during construction.

Method `fit()`: Initialize and fit a `solarModel` object given the specification contained in `$control`.

Usage:


```
solarModel$fit()
```

Returns: Updates the object state. The return value is not intended for use.

Method `fit_seasonal_model_Ct()`: Initialize and fit a [seasonalClearsky](#) model given the specification contained in `$control`.

Usage:

```
solarModel$fit_seasonal_model_Ct()
```

Returns: Updates the clear-sky model and fitted data stored in the object.

Method `compute_risk_drivers()`: Compute the risk drivers and impute the observation that are greater or equal to the clear sky level.

Usage:

```
solarModel$compute_risk_drivers()
```

Returns: Updates fitted data and outlier information stored in the object.

Method `fit_transform()`: Fit the parameters of the [solarTransform](#) object.

Usage:

```
solarModel$fit_transform(detect_outliers = TRUE)
```

Arguments:

`detect_outliers` Logical. If TRUE, transformation-bound outliers are detected.

Returns: Updates transformation parameters and transformed data stored in the object.

Method `fit_seasonal_mean()`: Fit a [seasonalModel](#) the transformed variable (Y_t) and compute deseasonalized series (Y_{t_tilde}).

Usage:

```
solarModel$fit_seasonal_mean()
```

Returns: Updates fitted seasonal mean quantities stored in the object.

Method `fit_monthly_mean()`: Correct the deseasonalized series (Y_{t_tilde}) by subtracting its monthly mean ($Y_{t_tilde_uncond}$).

Usage:

```
solarModel$fit_monthly_mean()
```

Returns: Updates monthly mean corrections when enabled.

Method `fit_mean_model()`: Fit an ARMA model (Y_{t_tilde}) and compute the residuals (ϵ_t).

Usage:

```
solarModel$fit_mean_model()
```

Returns: Updates ARMA fitted values and residuals stored in the object.

Method `fit_seasonal_variance()`: Fit a [seasonalModel](#) on AR squared residuals (ϵ_t) and compute deseasonalized residuals ϵ_{t_tilde} .

Usage:

```
solarModel$fit_seasonal_variance()
```

Returns: Updates seasonal variance quantities stored in the object.

Method `fit_monthly_variance()`: Correct the standardized series (ϵ_{t_tilde}) by subtracting its monthly mean (σ_{t_uncond}).

Usage:

```
solarModel$fit_monthly_variance()
```

Returns: Updates monthly variance corrections when enabled.

Method `correct_seasonal_variance()`: Correct the parameters of the seasonal variance to ensure a unitary variance

Usage:

```
solarModel$correct_seasonal_variance()
```

Returns: Updates seasonal variance parameters when correction is enabled.

Method `fit_variance_model()`: Fit a GARCH model on the deseasonalized residuals (`eps_tilde`). Compute the standardized (`u`) and monthly deseasonalized residuals (`u_tilde`).

Usage:

```
solarModel$fit_variance_model()
```

Returns: Updates GARCH variance and standardized residuals stored in the object.

Method `fit_mixture_model()`: Initialize and fit a solarMixture object.

Usage:

```
solarModel$fit_mixture_model()
```

Returns: Updates the monthly Gaussian mixture model stored in the object specification.

Method `update()`: Update the parameters inside object

Usage:

```
solarModel$update(params)
```

Arguments:

`params` List of parameters. See the slot `$coefficients` for a template.

Returns: Updates model parameters and resets the stored log-likelihood.

Method `update_moments()`: Update the moments inside object

Usage:

```
solarModel$update_moments()
```

Returns: Updates the stored conditional moments.

Method `update_logLik()`: Update the log-likelihood inside object

Usage:

```
solarModel$update_logLik()
```

Returns: Updates observation-level and total log-likelihood values.

Method `update_risk_drivers()`: Update the clear sky and risk drivers

Usage:

```
solarModel$update_risk_drivers()
```

Returns: Updates clear-sky values, risk drivers, and transformed data.

Method `update_classification()`: Update the classification of the Bernoulli random variable.

Usage:

```
solarModel$update_classification(filter = FALSE)
```

Arguments:

`filter` Logical. If TRUE, the mixture model is filtered before classification.

Returns: Updates mixture classification columns stored in fitted data.

Method `filter()`: Filter the time series when new parameters are supplied in the method `$update(params)`.

Usage:

```
solarModel$filter(fit = FALSE)
```

Arguments:

`fit` Logical. If TRUE, monthly mean and variance corrections are recomputed.

Returns: Updates the stored data summaries. The return value is not intended for use.

Method `logLik()`: Compute the log-likelihood of the model and update the slot `$loglik`.

Usage:

```
solarModel$logLik(moments, target = "Yt", quasi = FALSE)
```

Arguments:

`moments` Dataset containing the moments to use for computation.

`target` Character. Target variable to use, either "Yt" or "GHI".

`quasi` Logical. If TRUE, compute the Gaussian quasi log-likelihood.

Returns: A numeric vector of observation-level log-likelihood values.

Method `print()`: Print method for `solarModel` class.

Usage:

```
solarModel$print()
```

Returns: Prints a summary of the object.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
solarModel$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Note

Version 1.0.4

Examples

```
spec <- solarModel_spec$new()
spec$specification("Bologna", max_date = "2005-01-10")
model <- solarModel$new(spec)

if (interactive()) {
  model$fit()
  model$loglik
}
```

solarModel_AIC_BIC	<i>Compute the AIC and BIC of a solarModel object</i>
--------------------	---

Description

Compute the AIC and BIC of a solarModel object

Usage

```
solarModel_AIC_BIC(model, target = "GHI", type = "full")
```

Arguments

model	A fitted solarModel object.
target	Character. Target variable passed to the model log-likelihood.
type	Character. Data split used for the criteria. One of "full", "train", or "test".

Value

A tibble with model metadata, log-likelihood, parameter count, sample size, AIC, and BIC.

Note

Version 1.0.0.

solarModel_calibrate_theta	<i>Calibrate theta to match a certain level of solar radiation</i>
----------------------------	--

Description

Calibrate theta to match a certain level of solar radiation

Usage

```
solarModel_calibrate_theta(model, moments, e_RT_target, quiet = FALSE)
```

Arguments

e_RT_target	Numeric, vector of target expectation to match.
-------------	---

Details

Version 1.0.0.

solarModel_covariance *Compute the covariance*

Description

Compute the covariance

Usage

```
solarModel_covariance(  
  t_now,  
  mom_t,  
  mom_T,  
  GARCH,  
  NM_model,  
  theta = 0,  
  tol = 0.01  
)
```

Note

Version 1.0.0.

solarModel_forecast *Iterate the forecast on multiple dates*

Description

Iterate the forecast on multiple dates

Usage

```
solarModel_forecast(model, moments, ci = 0.1, lambda = 0)
```

Note

Version 1.0.0.

solarModel_grid	<i>Fit a grid of solarModels</i>
-----------------	----------------------------------

Description

Fit a grid of solarModels

Usage

```
solarModel_grid(
  spec,
  arOrder = 2,
  maOrder = 2,
  archOrder = 1,
  garchOrder = 1,
  QMLE = FALSE
)
```

Arguments

spec	A solarModel_spec object.
arOrder	Numeric. Maximum AR order.
maOrder	Numeric. Maximum MA order.
archOrder	Numeric. Maximum ARCH order.
garchOrder	Numeric. Maximum GARCH order.
QMLE	Logical. If TRUE, each fitted model is refitted with solarModel_QMLE().

Value

A tibble containing model names, selected orders, cloned specifications, and fitted solarModel objects.

Note

Version 1.0.0.

Examples

```
if (interactive()) {
  spec <- solarModel_spec$new()
  spec$specification("Bologna")
  models <- solarModel_grid(spec, 1, 1, 1, 1)
  models
}
```

`solarModel_match_params`*Match solarModel parameters in vector form*

Description

Match solarModel parameters in vector form

Usage

```
solarModel_match_params(vec_params, params)
```

Note

Version 1.0.0.

Examples

```
model = solarModel$new(spec)
model$fit()
vec_params <- c(theta = 1, alpha1 = 10)
solarModel_match_params(vec_params, model$coefficients)
```

`solarModel_PIT`*Compute the Probability Integral Transform (PIT)*

Description

Compute the Probability Integral Transform (PIT)

Usage

```
solarModel_PIT(model, moments)
```

Note

Version 1.0.4

solarModel_PIT_test	<i>Evaluate a KS test on the PIT.</i>
---------------------	---------------------------------------

Description

Evaluate a KS test on the PIT.

Usage

```
solarModel_PIT_test(
  model,
  type = "full",
  nyears,
  ci = 0.05,
  min_quantile = 0.015,
  max_quantile = 0.985
)
```

Arguments

model	An object of the class solarModel
type	character(1), type of test. Can be: "train" the test is performed only on train data; "test" the test is performed only on test data; "full" the test is performed only on the full data.
ci	p.value for rejection.
min_quantile	minimum quantile for the grid of values.
max_quantile	maximum quantile for the grid of values.

Details

See the functions [solarModel_PIT](#) and [ks_test](#) for more details.

Note

Version 1.0.0.

solarModel_predict	<i>Produce a forecast from a solarModel object</i>
--------------------	--

Description

Produce a forecast from a solarModel object

Usage

```
solarModel_predict(model, moments, lambda = 0, ci = 0.01)
```


Arguments

lambda Numeric scalar, Sugeno parameter.

Note

Version 1.0.0.

Examples

```
model = solarModel$new(spec)
model$fit()
moments <- model$moments$conditional[14,]
object <- solarModel_predict(model, moments, ci = 0.01)
object
```

solarModel_predict_plot

Plot a forecast from a solarModel object

Description

Plot a forecast from a solarModel object

Usage

```
solarModel_predict_plot(object, pdf_components = FALSE, type = "mix")
```

Examples

```
model = solarModel$new(spec)
model$fit()
df_n <- model$moments$conditional[23,]
solarModel_predict_plot(solarModel_predict(model, df_n, ci = 0.01))
```

solarModel_QMLE

QMLE Estimate

Description

QMLE Estimate

Usage

```
solarModel_QMLE(model, maxrestarts = 1, seed = 1, quiet = TRUE)
```

Note

Version 1.0.2

solarModel_spec	<i>Specification object for a solarModel</i>
-----------------	--

Description

R6 class that stores the component specifications and control settings used to initialize and fit a solarModel object.

Value

An R6 object of class solarModel_spec.

Public fields

outliers List containing detected outlier information.

place Character, optional name of the location considered.

coords A named list with coordinates for the selected location. Contains:

lat Numeric, reference latitude in degrees.

lon Numeric, reference longitude in degrees.

alt Numeric, reference altitude in metres.

Active bindings

target Character, name of the target variable to model. Can be "GHI" or "clearsky".

location A Tibble with the name and the coordinates of the location considered. Contains:

place Character, selected location

target Target variable

lat Numeric, reference latitude in degrees.

lon Numeric, reference longitude in degrees.

alt Numeric, reference altitude in metres.

model_name Character, standard model's name.

dates A named list with three sub-lists: data for the complete dataset, train for the training dataset, and test for the test dataset. Each sub-list is structured as follows:

from Character date, minimum date in the dataset.

to Character date, maximum date in the dataset.

nobs Integer scalar, number of observations contained in the dataset between from and to.

perc Numeric scalar, percentage of data in the dataset with respect to the complete data.

data Tibble, dataset with CAMS solar radiation data.

transform A [solarTransform](#) object with the transformation applied to the data.

seasonal_model_Ct A [seasonalClearsky](#) object with the clear-sky model applied to the data.

seasonal.mean A [seasonalModel](#) object with the seasonal model applied to the data.

mean.model An [ARMA_modelR6](#) object with the ARMA model applied to the data.

seasonal.variance A [seasonalModel](#) object with the seasonal variance applied to the data.

variance.model An [sGARCH](#) object with the GARCH model applied to the data.

`mixture.model` A `solarMixture` object with the monthly Gaussian Mixture models applied to the data.

`coefficients` Get the model parameters as a named list.

`std.errors` Get model parameter standard errors as a named list.

`hessian` Get the model hessian.

`jacobian` Get the model jacobian.

`clearsky_threshold` Numeric, parameter > 1, used to scale up CAMS clearsky.

`stochastic_clearsky` Logical, when TRUE the clear sky is considered stochastic.

`quiet` Logical. If TRUE, suppress messages where supported.

Methods

Public methods:

- `solarModel_spec$new()`
- `solarModel_spec$set_params()`
- `solarModel_spec$set_transform()`
- `solarModel_spec$set_clearsky()`
- `solarModel_spec$set_seasonal.mean()`
- `solarModel_spec$set_mean.model()`
- `solarModel_spec$set_seasonal.variance()`
- `solarModel_spec$set_variance.model()`
- `solarModel_spec$set_mixture.model()`
- `solarModel_spec$specification()`
- `solarModel_spec$update()`
- `solarModel_spec$print()`
- `solarModel_spec$clone()`

Method `new()`: Initialize a `solarModel_spec` object.

Usage:

```
solarModel_spec$new()
```

Returns: Initializes component specifications and control settings.

Method `set_params()`: Set general controls.

Usage:

```
solarModel_spec$set_params(
  stochastic_clearsky = FALSE,
  clearsky_threshold = 1.01,
  quiet = FALSE
)
```

Arguments:

`stochastic_clearsky` Logical, when TRUE the clear sky will be considered stochastic.

`clearsky_threshold` Numeric. Parameter greater than 1 used to scale up CAMS clear-sky radiation.

`quiet` Logical. If TRUE, suppress messages where supported.

Returns: Updates general control settings.

Method `set_transform()`: Control parameters for the solarTransform. See [solarTransform](#) for more details.

Usage:

```
solarModel_spec$set_transform(
  min_pos = 1,
  max_pos = 1,
  link = "invgumbel",
  delta = 0.05,
  threshold = 0.01
)
```

Arguments:

`min_pos` Integer, position of the minimum. For example when 2 the minimum is the second lowest value.

`max_pos` Integer, position of the maximum. For example when 3 the maximum is the third greatest value.

`link` Character, link function.

`delta` Numeric. Small adjustment used around fitted transformation bounds.

`threshold` Numeric. Threshold used to estimate the transformation parameters

α

and

β

. The default is 0.01. See [solarTransform](#) for more details.

Returns: Updates the stored solarTransform object and its controls.

Method `set_clearsky()`: List with specification's parameters of the clear sky model.

Usage:

```
solarModel_spec$set_clearsky(
  spec = seasonalClearsky_spec(),
  control = control_seasonalClearsky()
)
```

Arguments:

`spec` Named list, model's specification. See the function [seasonalClearsky_spec](#) for more details.

`control` Named list, control parameters. See the function [control_seasonalClearsky](#) for more details.

Returns: Updates the stored seasonalClearsky model object.

Method `set_seasonal.mean()`: List with specification's parameters of the seasonal mean $\bar{Y}_t$ for Y_t .

Usage:

```
solarModel_spec$set_seasonal.mean(
  order = 1,
  period = 365,
  include.trend = FALSE,
  include.intercept = TRUE,
  monthly.mean = FALSE
)
```

Arguments:

order Integer. Specify the order of the seasonal mean

$$\bar{Y}_t$$

. The default is 1.

period Integer, seasonal periodicity, the default is 365.

include.trend Logical. When TRUE an yearly trend

$$t$$

will be included in the seasonal model, otherwise will be excluded. The default is FALSE.

include.intercept Logical. When TRUE the intercept

$$a_0$$

will be included in the seasonal model, otherwise will be excluded. The default is TRUE.

monthly.mean Logical. When TRUE, monthly means are computed on the deseasonalized series

$$\tilde{Y}_t = Y_t - \bar{Y}_t$$

and subtracted from the series.

Returns: Updates the stored seasonal mean model.

Method `set_mean.model()`: List with specification's parameters of the ARMA model for deseasonalized series $\tilde{Y}_t = Y_t - \bar{Y}_t$.

Usage:

```
solarModel_spec$set_mean.model(
  arOrder = 1,
  maOrder = 0,
  include.intercept = FALSE
)
```

Arguments:

arOrder Integer. An integer specifying the order of the AR component. The default is 1.

maOrder Integer. An integer specifying the order of the MA component. The default is 0.

include.intercept Logical. When TRUE the intercept

$$\phi_0$$

will be included in the seasonal model, otherwise will be excluded. The default is FALSE.

Returns: Updates the stored ARMA mean model.

Method `set_seasonal.variance()`: List with specification's parameters of the seasonal variance $\bar{\sigma}_t$ for ARMA's residuals e_t

Usage:

```
solarModel_spec$set_seasonal.variance(
  order = 1,
  period = 365,
  include.trend = FALSE,
  correction = FALSE,
  monthly.mean = FALSE
)
```

Arguments:

`order` Integer. Specify the order of the seasonality of the seasonal variance. The default is 1.

`period` Integer, seasonal periodicity, the default is 365.

`include.trend` Logical. When TRUE an yearly trend

$$t$$

will be included in the seasonal model, otherwise will be excluded. The default is FALSE.

`correction` Logical. When TRUE, seasonal variance parameters are corrected to target unit variance for standardized residuals.

`monthly.mean` Logical. When TRUE a vector of 12 monthly std. deviations will be computed on the standardized residuals

$$\tilde{e}_t$$

and used to standardize the time series by month.

Returns: Updates the stored seasonal variance model.

Method `set_variance.model()`: List with specification's parameters of the GARCH variance σ_t for deseasonalized residuals $\tilde{e}_t = e_t/\bar{\sigma}_t$.

Usage:

```
solarModel_spec$set_variance.model(archOrder = 1, garchOrder = 1)
```

Arguments:

`archOrder` Integer. An integer specifying the order of the ARCH component. The default is 1.

`garchOrder` Integer. An integer specifying the order of the GARCH component. The default is 1.

Returns: Updates the stored GARCH variance model.

Method `set_mixture.model()`: List with specification's parameters of the Gaussian mixture model for GARCH residuals $u_t = \tilde{e}_t/\sigma_t$.

Usage:

```
solarModel_spec$set_mixture.model(
  abstol = 1e-20,
  match.expectation = FALSE,
  match.variance = FALSE,
  match.empiric = FALSE,
  method = "mclust",
  maxit = 5000,
  maxrestarts = 500
)
```

Arguments:

`abstol` Numeric. Absolute level for convergence of the EM-algorithm. The default is 1e-20.

`match.expectation` Logical. If TRUE, mixture parameters are fitted with an expectation-matching constraint.

`match.variance` Logical. If TRUE, mixture parameters are fitted with a variance-matching constraint.

`match.empiric` Logical, when TRUE and `match.expectation = TRUE` or `match.variance = TRUE` the mixture parameters will be estimated ensuring that mean and variance matches the empirical parameters. Otherwise if FALSE and `match.expectation = TRUE` or `match.variance = TRUE` the target expectation will be zero and the target variance 1.

method Character, package used to fit the parameters. Can be mclust or mixtools.
 maxit Integer. Maximum number of iterations for EM-algorithm. The default is 5000.
 maxrestarts Integer. Maximum number of restarts when EM-algorithm does not converge.
 The default is 500.

Returns: Updates the stored monthly Gaussian mixture model.

Method specification(): Specification function for a solarModel

Usage:

```
solarModel_spec$specification(
  place,
  target = "GHI",
  min_date,
  max_date,
  from,
  to,
  data
)
```

Arguments:

place Character, name of an element in the CAMS_data list.
 target Character, target variable to model. Can be GHI or clearsky.
 min_date Character. Date in the format YYYY-MM-DD. Minimum date for the complete data. If missing will be used the minimum data available.
 max_date Character. Date in the format YYYY-MM-DD. Maximum date for the complete data. If missing will be used the maximum data available.
 from Character. Date in the format YYYY-MM-DD. Starting date to use for training data. If missing will be used the minimum data available after filtering for min_date.
 to Character. Date in the format YYYY-MM-DD. Ending date to use for training data. If missing will be used the maximum data available after filtering for max_date.
 data Optional data frame for the selected location.

Returns: Updates stored data, location metadata, date ranges, and target variable.

Method update(): Update the parameters inside object

Usage:

```
solarModel_spec$update(params)
```

Arguments:

params List of parameters. See the slot \$coefficients for a template.

Returns: Updates component model parameters when params is supplied.

Method print(): Print method for solarModel_spec class.

Usage:

```
solarModel_spec$print()
```

Returns: Prints a summary of the specification.

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
solarModel_spec$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.3

Examples

```
spec <- solarModel_spec$new()
spec$specification("Bologna", max_date = "2005-01-10")
spec$model_name
```

solarModel_tests

*Autocorrelation and distribution tests***Description**

Evaluate a Kolmogorov-Smirnov test on the residuals of a solarModel model object against the estimated Gaussian mixture distribution and a Breush-pagan or Box-pierce test on the residuals.

Usage

```
solarModel_tests(
  model,
  lags = c(7),
  method = "bg",
  type = "full",
  ci = 0.05,
  min_quantile = 0.025,
  max_quantile = 0.985
)
```

Arguments

model	An object of the class solarModel
lags	Numeric vector. Lags on which perform the autocorrelation tests.
method	character(1), method of test performed. Can be: "bg" for Breush-Godfrey test; "bp" for Box-pierce test; "lb" for Ljung-Box test.
type	character(1), type of test. Can be: "train" the test is performed only on train data; "test" the test is performed only on test data; "full" the test is performed only on the full data.
ci	p.value for rejection.
min_quantile	minimum quantile for the grid of values.
max_quantile	maximum quantile for the grid of values.

Note

Version 1.0.0.

Examples

```
model = solarModel$new(spec)
model$fit()
solarModel_tests(model, train_data = TRUE)
```

solarModel_test_autocorr

Autocorrelation test

Description

Evaluate the autocorrelation in the components of a solarModel object.

Usage

```
solarModel_test_autocorr(
  model,
  nyears,
  method = "bg",
  type = "full",
  lag.max = 3,
  ci = 0.05
)
```

Arguments

model	An object of the class solarModel
method	character(1), method of test performed. Can be: "bg" for Breush-Godfrey test; "bp" for Box-pierce test; "lb" for Ljung-Box test.
type	character(1), type of test. Can be: "train" the test is performed only on train data; "test" the test is performed only on test data; "full" the test is performed only on the full data.
lag.max	integer(1), maximum lag to consider for the test.
ci	p.value for rejection.

Note

Version 1.0.0.

Examples

```
model = solarModel$new(spec)
model$fit()
solarModel_test_autocorr(model, method = "lb")
```

solarModel_test_distribution
Distribution test

Description

Evaluate a Kolmogorov-Smirnov test on the residuals of a solarModel model object against the estimated Gaussian mixture distribution.

Usage

```
solarModel_test_distribution(  
  model,  
  H0 = "gm",  
  type = "full",  
  ci = 0.05,  
  min_quantile = 0.025,  
  max_quantile = 0.985  
)
```

Arguments

model	An object of the class solarModel
H0	Character, null hypothesis for the residuals distribution. Can be: "gm" for a test against a gaussian mixture distribution; "norm" for a test against a gaussian distribution.
type	character(1), type of test. Can be: "train" the test is performed only on train data; "test" the test is performed only on test data; "full" the test is performed only on the full data.
ci	p.value for rejection.
min_quantile	minimum quantile for the grid of values.
max_quantile	maximum quantile for the grid of values.

Note

Version 1.0.0.

Examples

```
model = solarModel$new(spec)  
model$fit()  
solarModel_test_distribution(model)
```

`solarModel_test_forecast`*Compute metrics to test forecasts*

Description

Compute metrics to test forecasts

Usage

```
solarModel_test_forecast(model, ci = 0.1, type = c("train", "test", "full"))
```

Note

Version 1.0.0.

`solarModel_test_LPD`*Compute the Log-predictive density of a solarModel*

Description

Compute the Log-predictive density of a solarModel

Usage

```
solarModel_test_LPD(model, type = "full")
```

Note

Version 1.0.0.

`solarModel_test_pricing`*Compute metrics to test option pricing*

Description

Compute metrics to test option pricing

Usage

```
solarModel_test_pricing(  
  model,  
  type = c("train", "test", "full"),  
  control = control_solarOption()  
)
```

Note

Version 1.0.0.

solarModel_VaR	<i>Compute the Value at Risk and Expected Shortfall of a SolarModel</i>
----------------	---

Description

Compute the Value at Risk and Expected Shortfall of a SolarModel

Usage

```
solarModel_VaR(model, alpha = 0.05, ci = 0.05, ES = FALSE, type = "full")
```

Arguments

model	solarModel
alpha	Numeric vector of confidence levels. Allows for more than one alpha.
ci	Numeric scalar, confidence levels used to state if the Null is rejected or not on VaR tests.
ES	Logical, when TRUE the expected shortfall will be also computed for each alpha.
type	Numeric, type of evaluation, full on the complete data, train on the train data, test on the test data.

solarMoment	<i>Conditional moments for a solar model</i>
-------------	--

Description

Conditional moments for a solar model

Conditional moments for a solar model

Details

R6 class used to compute and store conditional moment quantities from a fitted solarModel object over a forecast horizon.

Version 1.0.1.

Value

An R6 object of class solarMoment.

Public fields

weights Tibble containing intermediate forecast weights and moment components.

X0 Numeric vector used as the initial ARMA state.

H0 Numeric vector used as the initial GARCH state.

t_now Date used as the conditioning date.

t_hor Date used as the forecast horizon.

place Character value identifying the reference place.

step Integer number of forecast steps.

Active bindings

`data` Tibble containing the stored moment summary for the horizon date.
`ARMA` List of stored ARMA components used in moment calculations.
`GARCH` List of stored GARCH components used in moment calculations.
`R_min_max` List containing stored minimum and maximum radiation bounds.
`M_Y` Numeric vector built from `M_Y1` and `M_Y0` in the stored data summary.
`S_Y` Numeric vector built from `S_Y1` and `S_Y0` in the stored data summary.
`probs` Numeric vector currently built from the stored `p1` value.

Methods**Public methods:**

- `solarMoment$new()`
- `solarMoment$filter()`
- `solarMoment$filter_ARMA()`
- `solarMoment$filter_NM()`
- `solarMoment$filter_GARCH()`
- `solarMoment$filter_weights()`
- `solarMoment$print()`
- `solarMoment$clone()`

Method `new()`: Initialize a `solarMoment` object.

Usage:

```
solarMoment$new(model, t_now, t_hor)
```

Arguments:

`model` A fitted `solarModel` object.

`t_now` Date or character value coercible to Date. Conditioning date.

`t_hor` Date or character value coercible to Date. Forecast horizon date.

Method `filter()`: Run the moment filtering steps.

Usage:

```
solarMoment$filter(theta = 0, B = NULL, t_cond = NULL)
```

Arguments:

`theta` Numeric value passed to `filter_NM()`.

`B` Optional vector or value passed to `filter_NM()`.

`t_cond` Optional date vector passed to `filter_NM()`.

Returns: Updates the object in place.

Method `filter_ARMA()`: Update ARMA forecast weights.

Usage:

```
solarMoment$filter_ARMA()
```

Returns: Updates the `weights` field in place.

Method `filter_NM()`: Update mixture moment components.

Usage:

```
solarMoment$filter_NM(theta = 0, B = NULL, t_cond = NULL)
```

Arguments:

theta Numeric value used to shift mixture component means.

B Optional vector or value used with t_cond to condition mixture weights.

t_cond Optional date vector identifying conditioning dates.

Returns: Updates the weights field in place.

Method filter_GARCH(): Update GARCH moment components.

Usage:

```
solarMoment$filter_GARCH()
```

Returns: Updates the weights field in place.

Method filter_weights(): Combine forecast weights into stored moment summaries.

Usage:

```
solarMoment$filter_weights()
```

Returns: Updates the weights field and active data summary in place.

Method print(): Print a summary of the conditional moment object.

Usage:

```
solarMoment$print()
```

Returns: Invisibly returns NULL.

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
solarMoment$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

solarMoments

Compute the generic conditional moments of a solarModel object

Description

Compute the generic conditional moments of a solarModel object

Usage

```
solarMoments(
  t_now,
  t_hor,
  data,
  ARMA,
  GARCH,
  NM_model,
  transform,
  theta = 0,
  quiet = FALSE
)
```

Arguments

t_now	Date for today.
t_hor	Horizon date.
data	Slot data of a solarModel object. See the function solarModel for details.
ARMA	Slot ARMA of a solarModel object.
GARCH	Slot GARCH of a solarModel object.
NM_model	Slot NM_model of a solarModel object.
transform	Slot transform of a solarModel object.
theta	Numeric, vector of shift parameters for the Gaussian mixture residuals.

Details

Version 1.0.0.

Examples

```

model = solarModel$new(spec)
model$fit()
t_now = "2019-07-11"
t_hor = "2019-10-19"
data = model$data
ARMA = model$ARMA
GARCH = model$GARCH
NM_model = model$NM_model
transform = model$transform
theta = 0
solarMoments(t_now, t_hor, data, ARMA, GARCH, NM_model, transform, theta = 0, quiet = FALSE)
filter(model$moments$conditional, date == t_hor)
filter(model$moments$unconditional, date == t_hor)

t_seq <- seq.Date(as.Date("2013-01-01"), as.Date("2013-12-31"), 1)
mom <- purrr::map_df(t_seq, ~solar::solarMoments(.x-150, .x, data, ARMA, GARCH, NM_model, transform, theta = 0))
solarOption_model(model, mom, control_options = control_solarOption(nyears = c(2005, 2024)))
solarOption_historical(model, control_options = control_solarOption(nyears = c(2005, 2024)))

```

solarMoments_conditional

Compute the conditional moments

Description

Compute the conditional moments

Usage

```
solarMoments_conditional(data, theta = 0)
```

Arguments

data	Slot data of a solarModel object. See the function solarModel for details.
theta	Numeric, shift parameter for the Gaussian mixture residuals.
control_model	An object with the class solarModel_spec. See the function solarModel_spec for details.

Details

Version 1.0.0.

solarMoments_path	<i>Condition the moments for a specific Bernoulli realization at a time t_{cond}</i>
-------------------	---

Description

Condition the moments for a specific Bernoulli realization at a time t_{cond}

Usage

```
solarMoments_path(moments, GARCH, NM_model, theta = 0, B = 1, t_cond)
```

Arguments

GARCH	Slot GARCH of a solarModel object.
NM_model	Slot NM_model of a solarModel object.
theta	Numeric, vector of shift parameters for the Gaussian mixture residuals.
B	conditioning value for the Bernoulli state at time thor
t_cond	conditioning date

Details

Version 1.0.0.

Examples

```
model = solarModel$new(spec)
model$fit()
GARCH = model$GARCH
NM_model = model$NM_model
# Compute the moments
moments <- model$Moments("2012-01-01", "2012-03-05")

# Condition the moments on a realization of B
moments
solarOption_model(model, moments)

mom_cond <- solarMoments_path(moments, GARCH, NM_model, theta = 0, B = 1, "2012-02-03")
solarOption_model(model, mom_cond)

mom_cond <- solarMoments_path(moments, GARCH, NM_model, theta = 0, B = 1, "2012-03-01")
```



```

solarOption_model(model, mom_cond)

mom_cond <- solarMoments_path(moments, GARCH, NM_model, theta = 0, B = 1, "2012-03-04")
solarOption_model(model, mom_cond)

mom_cond <- solarMoments_path(moments, GARCH, NM_model, theta = 0, B = 0, "2012-03-04")
solarOption_model(model, mom_cond)

mom_cond <- solarMoments_path(moments, GARCH, NM_model, theta = 0, B = c(0, 0), c("2012-03-03", "2012-03-04"))
solarOption_model(model, mom_cond)

mom_cond <- solarMoments_path(moments, GARCH, NM_model, theta = 0, B = c(1, 1), c("2012-03-03", "2012-03-04"))
solarOption_model(model, mom_cond)

```

solarMoments_unconditional

Compute the unconditional moments

Description

Compute the unconditional moments

Usage

```
solarMoments_unconditional(data, ARMA, GARCH, theta = 0)
```

Arguments

data	Slot data of a solarModel object. See the function solarModel for details.
ARMA	Slot ARMA of a solarModel object.
GARCH	Slot GARCH of a solarModel object.
theta	Numeric, shift parameter for the Gaussian mixture residuals.

Details

Version 1.0.0.

solarOption

Create a SoRad / SoREd contract specification

Description

Create a SoRad / SoREd contract specification

Create a SoRad / SoREd contract specification

Public fields

ticker description
 strike Strike price for solar radiation.
 t_pricing Character, pricing date.
 t_now Character, today date.
 t_init Character, inception date.
 t_hor Character, maturity date.
 tick Numeric, monetary conversion tick.
 contract_type Character, maturity date.

Active bindings

control control parameters
 tau Numeric, scalar. Time from t_now till t_hor in days.
 tau_accrued Numeric, scalar. Time from t_pricing till t_hor in days.

Methods**Public methods:**

- `solarOption$new()`
- `solarOption$set_contract()`
- `solarOption$set_control()`
- `solarOption$print()`
- `solarOption$clone()`

Method `new()`: Initialize the contract

Usage:

```
solarOption$new(contract_type = "SoRad")
```

Arguments:

contract_type Character, contract type "SoRad" or "SoREd"

Method `set_contract()`: Initialize the contract

Usage:

```
solarOption$set_contract(t_pricing, t_init, t_hor, strike, tick = 1)
```

Arguments:

t_pricing Character, pricing date.
 t_init Character, inception date.
 t_hor Character, maturity date.
 strike Numeric, strike price.
 tick Numeric monetary tick.

Method `set_control()`: Store a list of custom control parameters

Usage:

```
solarOption$set_control(control)
```

Arguments:

control List, control parameters.

Method print(): Print method

Usage:

```
solarOption$print()
```

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
solarOption$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.0.

solarOptionPayoff	<i>Structure the outputs of solarOption functions</i>
-------------------	---

Description

Structure the outputs of solarOption functions

Usage

```
solarOptionPayoff(data, leap_year = FALSE)
```

Arguments

data	df_payoff
leap_year	control params

Value

An object of the class solarOptionPayoff.

Note

Version 1.0.0.

solarOption_calibrate_theta

Calibrate the time series of theta to match a certain level of Option price

Description

Calibrate the time series of theta to match a certain level of Option price

Usage

```
solarOption_calibrate_theta(
  model,
  moments,
  P_target,
  put = TRUE,
  quiet = FALSE,
  control_options = control_solarOption()
)
```

Arguments

model	An object with the class solarModel. See the function solarModel for details.
P_target	Numeric, vector of target prices to match.
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.

Details

Version 1.0.0.

solarOption_choquet *Compute Choquet price for a Solar Option*

Description

Compute Choquet price for a Solar Option

Usage

```
solarOption_choquet(
  model,
  moments,
  portfolio,
  nmonths = 1:12,
  lambda = 0,
```

```

implvol = 1,
put = TRUE,
control_options = control_solarOption()
)

```

Arguments

model	An object with the class solarModel. See the function solarModel for details.
moments	Tibble containing the forecasted moments for different days ahead. See the function solarMoments for more details.
portfolio	Optional, A list of objects of the class solarOptionPortfolio.
nmonths	Numeric vector, months in which the payoff should be computed. Can vary from 1 (January) to 12 (December).
lambda	Numeric, Sugeno parameter.
implvol	Numeric, implied volatility.
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.

Note

Version 1.0.0.

Examples

```

model = solarModel$new(spec)
model$fit()
moments <- model$moments$unconditional[1:365,]
lambda = 0
control_options = control_solarOption()
solarOption_choquet(model, moments[1:30,], lambda = 0.01)
solarOption_model(model, moments[1,])
solarOption_choquet(model, moments, lambda = 0.1, put = F)

```

solarOption_discount *Discount factor of a Solar Option*

Description

Discount factor of a Solar Option

Usage

```
solarOption_discount(tau, P = 1, Gamma_h = 0, r = 0.03/365)
```

Arguments

tau	Numeric, time to maturity in days.
P	Numeric, price of the contract.
Gamma_h	Numeric, hedged payoff.
r	Numeric, daily risk-free rate.

Details

The discount factor reads:

$$B(\tau, P, \Gamma^h, r) = e^{-r\tau} + \frac{\Gamma^h}{P}(1 - e^{-r\tau})$$

Note

Version 1.0.0.

Examples

```
solarOption_discount(365. 0.6, 2, 0.000008)
solarOption_discount(365, 0.3, 2, 0.00008)
```

solarOption_greeks	<i>Compute the Greeks of a Solar Option</i>
--------------------	---

Description

Compute the Greeks of a Solar Option

Usage

```
solarOption_greeks(model, moments, put = TRUE, elasticities = FALSE)
```

Note

Version 1.0.0.

Examples

```
# Model fit
model <- solarModel$new(spec)
model$fit()
solarOption_greeks(model, model$Moments("2023-01-01", "2024-05-01"))
```

solarOption_historical

Payoff of solar options on historical data

Description

Payoff of solar options on historical data

Usage

```
solarOption_historical(
  model,
  data = NULL,
  nmonths = 1:12,
  put = TRUE,
  control_options = control_solarOption()
)
```

Arguments

model	An object with the class solarModel. See the function solarModel for details.
data	Optional dataset for the computation of the payoff.
nmonths	Numeric vector, months in which the payoff should be computed. Can vary from 1 (January) to 12 (December).
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.

Value

An object of the class solarOptionPayoff.

Note

Version 1.0.0.

Examples

```
# Solar model
model <- solarModel$new(spec)
model$fit()
# Historical payoff (put)
solarOption_historical(model, put=TRUE)
# Historical payoff (call)
solarOption_historical(model, put=FALSE)
```

solarOption_index_greeks

Compute the Greeks of a Solar Option index

Description

Compute the Greeks of a Solar Option index

Usage

```
solarOption_index_greeks(model, moments, elasticities = FALSE)
```

Note

Version 1.0.0.

Examples

```
# Model fit
model <- solarModel$new(spec)
model$fit()

t_now <- as.Date("2024-01-01")
t_hor <- as.Date("2024-12-31")
dates <- seq.Date(t_now+1, t_hor, 1)
mom <- purrr::map_df(dates, ~model$Moments(t_now, .x))
solarOption_index_greeks(model, mom, elasticities = TRUE)
mom_test <- mom
mom_test$beta <- mom_test$beta*1.01
solarOption_index_greeks(model, mom_test, elasticities = TRUE)
```

solarOption_lambda

Calibrate the implied Choquet parameter

Description

Calibrate the implied Choquet parameter

Usage

```
solarOption_lambda(
  model,
  moments,
  P_target,
  r_imp = 0,
  put = TRUE,
  quiet = FALSE,
  control_options = control_solarOption()
)
```


Arguments

model	An object with the class solarModel. See the function solarModel for details.
moments	Tibble containing the forecasted moments for different days ahead. See the function solarMoments for more details.
P_target	Numeric, target price to match.
r_imp	Numeric, implied return from measure under P.
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.

Details

The optimization function will find the best parameter λ such that

$$\underset{-0.5 < \lambda < 1}{\operatorname{argmin}} \left(P^{\mathbb{P}(\lambda)}(1 + r^{\text{imp}}) - P_{\text{target}} \right)^2$$

Note

Version 1.0.0.

solarOption_model	<i>Compute the premium given the moments</i>
-------------------	--

Description

Compute the premium given the moments

Usage

```
solarOption_model(
  model,
  moments,
  portfolio,
  nmonths = 1:12,
  lambda = 0,
  implvol = 1,
  put = TRUE,
  control_options = control_solarOption()
)
```

Arguments

model	An object with the class solarModel. See the function solarModel for details.
moments	Tibble containing the forecasted moments for different days ahead. See the function solarMoments for more details.
portfolio	Optional, A list of objects of the class solarOptionPortfolio.

nmonths	Numeric vector, months in which the payoff should be computed. Can vary from 1 (January) to 12 (December).
lambda	Numeric, Sugeno parameter.
implvol	Numeric, implied volatility.
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.

Note

Version 1.0.0.

Examples

```
# Model
model = solarModel$new(spec)
model$fit()
# Pricing without portfolio
moments <- model$moments$unconditional
# Premium
premium_Vt <- solarOption_model(model, moments, put = TRUE)
# Pricing date
t_now <- as.Date("2021-12-31")
# Inception date
t_init <- as.Date("2022-01-01")
# Maturity date
t_hor <- as.Date("2022-12-31")
# solar options portfolio
portfolio <- SoRadPortfolio(model, t_now, t_init, t_hor)
# Moments
moments <- purrr::map_df(portfolio, ~model$Moments(t_now, .x$t_hor))
# Premium
premium_Vt <- solarOption_model(model, moments, portfolio, put = TRUE)
premium_Vt$payoff_year$premium
```

solarOption_moments	<i>Compute the first fourth moments and variance of a SoRad</i>
---------------------	---

Description

Compute the first fourth moments and variance of a SoRad

Usage

```
solarOption_moments(
  moments,
  transform,
  lambda = 0,
  put = TRUE,
  link = "invgumbel",
  control_options = control_solarOption()
)
```

Arguments

transform	slot transform of a solarModel
lambda	Sugeno parameter
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.

Note

Version 1.0.0.

Examples

```
# Solar model
model <- solarModel$new(spec)
model$fit()
moments <- model$moments$unconditional
solarOption_moments(moments[1:365,], model$transform)
```

solarOption_payoff	<i>Payoff function of a Solar Option</i>
--------------------	--

Description

Payoff function of a Solar Option

Usage

```
solarOption_payoff(R, K = 0, put = TRUE)
```

Arguments

R	Numeric, vector of values of solar radiation at maturity.
K	Numeric, scalar or vector of strikes.
put	Logical, when TRUE, the default, the function will return the output of a put payoff otherwise a call payoff. See the details.

Details

When put = TRUE, the function evaluate a put payoff:

$$(K - R)^+ = (K - R)1_{K > R}$$

When put = FALSE, the function evaluate a call payoff:

$$(R - K)^+ = (R - K)1_{R \geq K}$$

Note

Version 1.0.0.

Examples

```
# Put
solarOption_payoff(10, 9, put = TRUE)
solarOption_payoff(8, 9, put = TRUE)
# Call
solarOption_payoff(10, 9, put = FALSE)
solarOption_payoff(8, 9, put = FALSE)
```

solarOption_price	<i>Compute the price of a solarOption</i>
-------------------	---

Description

Compute the price of a solarOption

Usage

```
solarOption_price(K, pdf_R, R_min, R_max, put = TRUE)
```

Arguments

K	Numeric, strike price.
pdf_R	Function, density function of solar radiation at maturity.
R_min	Numeric, minimum value of solar radiation at maturity.
R_max	Numeric, maximum value of solar radiation at maturity.

Details

When put = TRUE, the function evaluate a put payoff:

$$\mathbb{E}\{(K - R)^+\} = K\mathbb{P}\{R_T < K\} - \mathbb{E}\{R1_{K > R_T}\}$$

When put = FALSE, the function evaluate a call payoff:

$$\mathbb{E}\{(K - R)^+\} = \mathbb{E}\{R1_{K < R_T}\} - K\mathbb{P}\{R_T > K\}$$

Note

Version 1.0.0.

Examples

```
# Parameters for the pdf of Rt
Ct <- 7
alpha <- 0.01
beta <- 0.92
Rt_min_max <- Ct * c(1-alpha-beta, 1-alpha)
pdf_R <- function(x) dsolarGHI(x, Ct, alpha, beta, function(x) dnorm(x))
cdf_R <- function(x) psolarGHI(x, Ct, alpha, beta, function(x) pnorm(x))
# Put price
solarOption_price(5, R_min = Rt_min_max[1], R_max = Rt_min_max[2], TRUE)
# Call price
solarOption_price(5, pdf_R, Rt_min_max[1], Rt_min_max[2], FALSE)
```

solarOption_pricing	<i>Compute the price of a solarOption</i>
---------------------	---

Description

Compute the price of a solarOption

Usage

```
solarOption_pricing(
  moments,
  sorad,
  lambda = 0,
  put = TRUE,
  link = "invgumbel",
  control_options = control_solarOption()
)
```

Arguments

moments	Tibble containing the forecasted moments for different days ahead. See the function solarMoments for more details.
sorad	An object of the class solarOption.
lambda	Numeric, Sugeno parameter.
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
control_options	Named list with control parameters. See control_solarOption for more details.

Note

Version 1.0.0.

Examples

```
# Model
model = solarModel$new(spec)
model$fit()
moments <- filter(model$moments$conditional, Year == 2022)
# Pricing without contracts
solarOption_pricing(moments[1,])
# Pricing with contracts specification
sorad <- solarOption$new()
sorad$set_contract("2021-12-31", "2022-01-01", "2022-04-20", moments$GHI_bar[1])
solarOption_pricing(moments[1,], sorad)
solarOption_pricing(moments[1,], sorad, lambda = 0.02)
solarOption_pricing(moments[1,], sorad, lambda = -0.02)
```

solarOption_scenario	<i>Compute average premium on simulated Data</i>
----------------------	--

Description

Compute average premium on simulated Data

Usage

```
solarOption_scenario(
  model,
  scenario,
  nmonths = 1:12,
  put = TRUE,
  nsim,
  control_options = control_solarOption()
)
```

Arguments

model	An object with the class solarModel. See the function solarModel for details.
scenario	object with the class solarModelScenario. See the function solarModel_scenarios for details.
nmonths	Numeric vector, months in which the payoff should be computed. Can vary from 1 (January) to 12 (December).
put	Logical, when TRUE, the default, will be computed the price for a put contract, otherwise for a call contract.
nsim	number of simulation to use for computation.
control_options	control function, see control_solarOption for details.

Value

An object of the class solarOptionPayoff.

Note

Version 1.0.2

Examples

```
# Solar model
model <- solarModel$new(spec)
model$fit()
# Simulate scenarios
scenario <- solarScenario_by(model, from = "2011-01-01", to = "2012-01-01", by = "1 month", nsim = 10, seed = 3)

solarOption_scenario(model, scenario)
solarOption_historical(model)
solarScenario_plot(scenario)
```

solarScenario	<i>Solar scenario simulation</i>
---------------	----------------------------------

Description

Solar scenario simulation

Solar scenario simulation

Details

R6 class used to simulate residuals and store filtered scenario paths from a solarScenario_spec object.

Public fields

seed Integer seed used when simulating residuals.

residuals Tibble or list-like object containing simulated residuals waiting to be filtered.

Active bindings

spec Stored solarScenario_spec object.

Methods**Public methods:**

- `solarScenario$new()`
- `solarScenario$simulate_residuals()`
- `solarScenario$filter()`
- `solarScenario$clone()`

Method `new()`: Initialize a solarScenario object.

Usage:

```
solarScenario$new(simSpec, seed = 1)
```

Arguments:

simSpec A solarScenario_spec object.
 seed Integer seed used for residual simulation.

Method simulate_residuals(): Simulate residuals for scenario generation.

Usage:

```
solarScenario$simulate_residuals(nsim = 1)
```

Arguments:

nsim Integer number of residual paths to simulate.

Returns: Updates the residuals field.

Method filter(): Filter simulated residuals into scenario paths.

Usage:

```
solarScenario$filter(all = FALSE)
```

Arguments:

all Logical. If TRUE, filter all stored residuals; otherwise, filter residuals marked for filtering.

Returns: Appends filtered scenario paths to spec\$simulations and updates residual flags.

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
solarScenario$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.3

solarScenario_by	<i>Simulate multiple scenarios</i>
------------------	------------------------------------

Description

Simulate multiple scenarios of solar radiation with a solarModel object.

Usage

```
solarScenario_by(
  model,
  from = "2010-01-01",
  to = "2011-01-01",
  by = "1 year",
  theta = 0,
  nsim = 1,
  seed = 1,
  quiet = FALSE
)
```


Arguments

model	object with the class solarModel. See the function solarModel for details.
from	character, start Date for simulations in the format YYYY-MM-DD.
to	character, end Date for simulations in the format YYYY-MM-DD.
by	character, steps for multiple scenarios, e.g. 1 day (day-ahead simulations), 15 days, 1 month, 3 months, ecc. For each step are simulated nsim scenarios.
theta	numeric, shift parameter for the mixture.
nsim	integer, number of simulations.
seed	scalar integer, starting random seed.
quiet	logical

Note

Version 1.0.1

Examples

```
# Solar Model
model <- solarModel$new(spec)
model$fit()
scenario <- solarScenario_by(model, "2005-01-10", "2020-01-01", theta = 0, nsim = 4, by = "1 year")
# Plot
solarScenario_plot(scenario, nsim = 2)
# Solar Option
solarOption_scenario(model, scenario)
solarOption_historical(model)
```

solarScenario_filter *Filter*

Description

Specification of scenarios of a solarModel

Usage

```
solarScenario_filter(spec, residuals)
```

Note

Version 1.0.2

Examples

```
model <- solarModel$new(spec)
model$fit()
# Specification
spec <- solarScenario_spec$new(model)
residuals <- solarScenario_residuals(spec, 4, 1)
solarScenario_filter(spec, residuals)
```

solarScenario_plot	<i>Plot scenarios from a solarScenario object</i>
--------------------	---

Description

Plot scenarios from a solarScenario object

Usage

```
solarScenario_plot(  
  scenario,  
  target = "GHI",  
  nsim = 10,  
  empiric = TRUE,  
  ci = 0.05  
)
```

Examples

```
# solarModel  
model <- solarModel$new(spec)  
model$fit()  
# Scenarios  
scenario <- solarScenario(model, "2016-01-01", "2017-01-01", nsim = 10, by = "1 month")  
# Plot  
solarScenario_plot(scenario)
```

solarScenario_residuals	<i>Simulate</i>
-------------------------	-----------------

Description

Specification of scenarios of a solarModel

Usage

```
solarScenario_residuals(spec, nsim = 1, seed = 1)
```

Note

Version 1.0.2

Examples

```
model <- solarModel$new(spec)  
model$fit()  
# Specification  
spec <- solarScenario_spec$new(model)  
solarScenario_residuals(spec, 4, 1)
```

solarScenario_spec	<i>Scenario specification</i>
--------------------	-------------------------------

Description

Scenario specification

Scenario specification

Details

R6 class used to store the data and model components needed to simulate scenarios from a fitted solarModel object.

Public fields

location Tibble containing the name of the reference location and its coordinates.

target Character, target variable.

quiet Logical. If TRUE, suppress selected messages and warnings.

control Named list with date range and simulation setup values.

simulations List of simulated scenario data sets.

Active bindings

place Character value with the reference place.

ARMA List of stored ARMA model parameters used for simulation.

GARCH List of stored GARCH model parameters used for simulation.

transform Transformation object copied from the model specification.

data_sim Tibble containing the data used as the simulation template.

emp Tibble containing empirical data stored in the specification.

nsim Integer number of stored simulations.

scenarios Nested tibble built from stored simulations.

Methods**Public methods:**

- `solarScenario_spec$new()`
- `solarScenario_spec$prob()`
- `solarScenario_spec$print()`
- `solarScenario_spec$clone()`

Method `new()`: Initialize a solarScenario_spec object.

Usage:

```
solarScenario_spec$new(
  model,
  from = "2010-01-01",
  to = "2010-12-31",
  theta = 0,
  exclude_known = FALSE,
  seed = 1,
  quiet = FALSE
)
```

Arguments:

model A fitted solarModel object.

from Date or character value coercible to Date. First date of the scenario period.

to Date or character value coercible to Date. Last date of the scenario period.

theta Numeric value stored in the simulation data as theta.

exclude_known Logical. If TRUE, empirical data stored in the specification are restricted to dates from *from* to *to*.

seed Integer seed stored for scenario simulation.

quiet Logical. If TRUE, suppress selected messages and warnings.

Method `prob()`: Marginal mixture probabilities.

Usage:

```
solarScenario_spec$prob(nmonth)
```

Arguments:

nmonth Integer vector of month numbers.

Returns: A numeric vector or object returned by the stored probability model's `predict()` method.

Method `print()`: Print a summary of the scenario specification.

Usage:

```
solarScenario_spec$print()
```

Returns: Invisibly returns NULL.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
solarScenario_spec$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

Note

Version 1.0.3

solarScenario_VaR	<i>Compute the Value at Risk from simulated values</i>
-------------------	--

Description

Compute the Value at Risk from simulated values

Usage

```
solarScenario_VaR(scenario, alpha = 0.05)
```

Arguments

alpha	Confidence level for the VaR
scenarios	An object of the class solarScenario

Note

Version 1.0.0.

Examples

```
# solarModel
model <- solarModel$new(spec)
model$fit()
# Scenarios
scenario <- solarScenario(model, "2016-01-01", "2017-01-01", nsim = 10, by = "1 month")
# VaR
solarScenario_VaR(scenario, 0.05)
```

solarSpectra	<i>Solar spectral data</i>
--------------	----------------------------

Description

A tabular data set of spectral irradiance values.

Usage

```
data(solarSpectra)
```

Format

A tibble with 2,002 rows and 4 numeric columns:

Lambda Wavelength values.
Am0 Numeric spectral values.
Am1 Numeric spectral values.
Am2 Numeric spectral values.

solarTransform	<i>Solar Model transformation functions</i>
----------------	---

Description

Solar Model transformation functions

Solar Model transformation functions

Super class

`solarrr::boundTransform -> solarTransform`

Public fields

`control` List to contain custom control parameters.

`epsilon` Numeric, ϵ transformation parameter.

Methods

Public methods:

- `solarTransform$new()`
- `solarTransform$X()`
- `solarTransform$iX()`
- `solarTransform$eta()`
- `solarTransform$ieta()`
- `solarTransform$RY()`
- `solarTransform$iRY()`
- `solarTransform$clone()`

Method `new()`: Initialize a `solarTransform` object.

Usage:

```
solarTransform$new(alpha = 0, beta = 1, link = "invgumbel")
```

Arguments:

`alpha` Numeric, α transformation parameter.

`beta` Numeric, β transformation parameter.

`link` Character, link function.

Method `X()`: Map the solar radiation R_t in the risk driver X_t .

Usage:

```
solarTransform$X(Rt, Ct)
```

Arguments:

`Rt` Numeric, solar radiation $R_t \in [C_t(1 - \alpha - \beta), C_t(1 - \alpha)]$.

`Ct` Numeric, clear sky radiation.

Details: The function computes:

$$X(R_t) = 1 - R_t/C_t$$

Returns: Numeric, risk driver $X_t \in (\alpha, \alpha + \beta)$.

Method `ix()`: Map the risk driver X_t in solar radiation R_t

Usage:

`solarTransform$ix(Xt, Ct)`

Arguments:

`Xt` Numeric, risk driver in $X_t \in (\alpha, \alpha + \beta)$.

`Ct` Numeric, clear sky radiation.

Details: The function computes:

$$X^{-1}(X_t) = C_t(1 - X_t)$$

Returns: Numeric, solar radiation $R_t \in [C_t(1 - \alpha - \beta), C_t(1 - \alpha)]$.

Method `eta()`: Map the solar radiation R_t in the normalized variable X'_t .

Usage:

`solarTransform$eta(Rt, Ct)`

Arguments:

`Rt` Numeric, solar radiation $R_t \in [C_t(1 - \alpha - \beta), C_t(1 - \alpha)]$.

`Ct` Numeric, clear sky radiation.

Details: The function computes:

$$\eta(R_t) = \frac{1}{\beta}(1 - \alpha - R_t/C_t)$$

Returns: Numeric, normalized risk driver $X'_t \in (0, 1)$.

Method `ieta()`: Map the normalized variable X'_t to the solar radiation R_t .

Usage:

`solarTransform$ieta(Xt_prime, Ct)`

Arguments:

`Xt_prime` Numeric, normalized risk driver $X'_t \in (0, 1)$.

`Ct` Numeric, clear sky radiation.

Details: The function computes:

$$\eta^{-1}(X'_t) = C_t(1 - \alpha - \beta \cdot X'_t)$$

Returns: Numeric, solar radiation $R_t \in [C_t(1 - \alpha - \beta), C_t(1 - \alpha)]$.

Method `RY()`: Convert solar radiation R_t into the transformed variable Y_t .

Usage:

`solarTransform$RY(Rt, Ct)`

Arguments:

`Rt` Numeric, solar radiation $R_t \in [C_t(1 - \alpha - \beta), C_t(1 - \alpha)]$.

`Ct` Numeric, clear sky radiation.

Details: The function computes:

$$RY(R_t) = g\left(\frac{1}{\beta}(1 - \alpha - R_t/C_t)\right)$$

Returns: Transformed variable $Y_t \in (-\infty, \infty)$.

Method `iRY()`: Convert the transformed variable Y_t into solar radiation R_t .

Usage:

```
solarTransform$iRY(Yt, Ct)
```

Arguments:

`Yt` Numeric, transformed variable $Y_t \in (-\infty, \infty)$.

`Ct` Numeric, clear sky radiation.

Details: The function computes:

$$\text{iRY}(Y_t) = C_t(1 - \alpha - \beta g^{-1}(Y_t))$$

Returns: Numeric, solar radiation $R_t \in [C_t(1 - \alpha - \beta), C_t(1 - \alpha)]$.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
solarTransform$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Note

Version 1.0.0.

Examples

```
st <- solarTransform$new()
```

SoRadPorfolio

Create a Solar Option Index portfolio

Description

Create a Solar Option Index portfolio

Usage

```
SoRadPorfolio(model, t_now, t_init, t_hor)
```

Note

Version 1.0.0.

spectralDistribution	<i>Compute the spectral distribution for a black body</i>
----------------------	---

Description

Compute the spectral distribution for a black body

Usage

```
spectralDistribution(x, measure = "nanometer")
```

Arguments

x	Numeric vector. Wavelength values. Values are interpreted in nanometers when measure = "nanometer" and in micrometers when measure = "micrometer".
measure	Character. Unit used for the wavelength scale and returned spectral density. One of "nanometer" or "micrometer".

Value

A numeric vector of spectral density values.

Examples

```
spectralDistribution(c(500, 600))  
spectralDistribution(c(0.5, 0.6), measure = "micrometer")
```

sugeno_bounds	<i>Sugeno upper and lower parameters.</i>
---------------	---

Description

Sugeno upper and lower parameters.

Usage

```
sugeno_bounds(lambda)
```

Arguments

lambda	Numeric, distortion parameter.
--------	--------------------------------

Value

A named list with positive, average, and negative parameter values.

VaR_test	<i>Bernoulli test on the violations of the VaR</i>
----------	--

Description

Bernoulli test on the violations of the VaR

Usage

```
VaR_test(et, VaR, ci = 0.01)
```

Arguments

et	vector or matrix of violations. On the columns different VaR on the rows observations.
VaR	vector of values at risk
ci	confidence level for the test

VaR_viol	<i>Violations of the VaR</i>
----------	------------------------------

Description

Violations of the VaR

Usage

```
VaR_viol(x, q)
```

Arguments

x	vector, realized data
q	vector or matrix, on the columns different VaR on the rows observations.

Index

- * **ARMA-moments**
 - ARMA_covariance, [5](#)
 - ARMA_expectation, [6](#)
 - ARMA_filter, [6](#)
 - ARMA_forecast, [7](#)
 - ARMA_next_step, [11](#)
 - ARMA_variance, [13](#)
- * **ARMA-params**
 - AR_params_to_phi, [15](#)
 - AR_params_to_zeta, [15](#)
 - ARMA_companion_matrix, [4](#)
 - ARMA_params_to_zeta, [12](#)
 - ARMA_vector_b, [14](#)
- * **ARMA**
 - AR_params_to_phi, [15](#)
 - AR_params_to_zeta, [15](#)
 - ARMA_companion_matrix, [4](#)
 - ARMA_covariance, [5](#)
 - ARMA_expectation, [6](#)
 - ARMA_filter, [6](#)
 - ARMA_forecast, [7](#)
 - ARMA_modelR6, [9](#)
 - ARMA_next_step, [11](#)
 - ARMA_params_to_zeta, [12](#)
 - ARMA_variance, [13](#)
 - ARMA_vector_b, [14](#)
- * **GARCH**
 - GARCH_companion_matrix, [46](#)
 - sGARCH, [71](#)
- * **clearsky**
 - clearsky_delta, [23](#)
 - clearsky_optimizer, [23](#)
 - clearsky_optimizer_CLS, [24](#)
 - clearsky_optimizer_delta, [25](#)
 - clearsky_optimizer_RLS, [25](#)
 - clearsky_outliers, [26](#)
 - control_seasonalClearsky, [27](#)
 - seasonalClearsky, [58](#)
 - seasonalClearsky_spec, [60](#)
- * **control**
 - control_solarHedging, [28](#)
 - control_solarOption, [28](#)
- * **datasets**
 - Bologna, [16](#)
 - CAMS_data, [22](#)
 - solarSpectra, [125](#)
- * **diagnostic**
 - PIT, [57](#)
 - PIT_test, [57](#)
 - VaR_test, [130](#)
 - VaR_viol, [130](#)
- * **distributions**
 - desscher, [29](#)
 - desscherMixture, [30](#)
 - dgumbel, [32](#)
 - dinvgumbel, [33](#)
 - dkumaraswamy, [34](#)
 - dmix2norm, [35](#)
 - dmixnorm, [37](#)
 - dsnrm, [38](#)
 - dsolarGHI, [39](#)
 - dsolarK, [41](#)
 - dsolarX, [43](#)
 - dsugeno, [44](#)
 - dtnorm, [45](#)
 - mix2norm_M_Yl_given_Yv, [53](#)
 - mix2norm_probs_given_Yv, [53](#)
 - mix2norm_S_Yl_given_Yv, [54](#)
- * **gaussianMixture**
 - gaussianMixture, [47](#)
 - solarMixture, [75](#)
- * **seasonalModel**
 - seasonalModel, [61](#)
- * **solarHedging**
 - solarHedging_model, [73](#)
 - solarHedging_scenarios, [74](#)
- * **solarModel_test**
 - solarModel_test_pricing, [99](#)
- * **solarModel**
 - solarModel, [79](#)
 - solarModel_AIC_BIC, [84](#)
 - solarModel_calibrate_theta, [84](#)
 - solarModel_covariance, [85](#)
 - solarModel_forecast, [85](#)
 - solarModel_grid, [86](#)
 - solarModel_match_params, [87](#)

- solarModel_PIT, [87](#)
- solarModel_PIT_test, [88](#)
- solarModel_predict, [88](#)
- solarModel_QMLE, [89](#)
- solarModel_spec, [90](#)
- solarModel_test_autocorr, [97](#)
- solarModel_test_distribution, [98](#)
- solarModel_test_forecast, [99](#)
- solarModel_test_LPD, [99](#)
- solarModel_tests, [96](#)
- solarTransform, [126](#)
- * solarMoments**
 - solarMoment, [100](#)
 - solarMoments, [102](#)
 - solarMoments_conditional, [103](#)
 - solarMoments_path, [104](#)
 - solarMoments_unconditional, [105](#)
- * solarOption**
 - solarOption, [105](#)
 - solarOption_calibrate_theta, [108](#)
 - solarOption_choquet, [108](#)
 - solarOption_discount, [109](#)
 - solarOption_greeks, [110](#)
 - solarOption_historical, [111](#)
 - solarOption_index_greeks, [112](#)
 - solarOption_lambda, [112](#)
 - solarOption_model, [113](#)
 - solarOption_moments, [114](#)
 - solarOption_payoff, [115](#)
 - solarOption_price, [116](#)
 - solarOption_pricing, [117](#)
 - solarOption_scenario, [118](#)
 - solarOptionPayoff, [107](#)
 - SoRadPorfolio, [128](#)
- * solarScenario**
 - solarScenario, [119](#)
 - solarScenario_by, [120](#)
 - solarScenario_filter, [121](#)
 - solarScenario_residuals, [122](#)
 - solarScenario_spec, [123](#)
 - solarScenario_VaR, [125](#)
- * utils**
 - boundTransform, [16](#)
 - boundTransform_av_links, [20](#)
 - boundTransform_link.gumbel, [20](#)
 - boundTransform_link.identity, [21](#)
 - boundTransform_link.invgumbel, [21](#)
 - boundTransform_link.logis, [21](#)
 - boundTransform_link.norm, [22](#)
- AR_params_to_phi, [4](#), [13–15](#), [15](#)
- AR_params_to_zeta, [4](#), [13–15](#), [15](#)
- ARMA_companion_matrix, [4](#), [5–7](#), [9](#), [12–15](#)
- ARMA_covariance, [5](#), [6–8](#), [12](#), [13](#)
- ARMA_expectation, [5](#), [6](#), [7](#), [8](#), [10](#), [12](#), [13](#)
- ARMA_filter, [5](#), [6](#), [6](#), [8](#), [10](#), [12](#), [13](#)
- ARMA_fit, [7](#)
- ARMA_forecast, [5–7](#), [7](#), [12](#), [13](#)
- ARMA_loss_CSS, [8](#)
- ARMA_loss_logLik, [8](#)
- ARMA_modelR6, [9](#), [90](#)
- ARMA_next_step, [5–8](#), [10](#), [11](#), [13](#)
- ARMA_params_to_zeta, [4](#), [12](#), [14](#), [15](#)
- ARMA_variance, [5–8](#), [10](#), [12](#), [13](#)
- ARMA_vector_b, [4–7](#), [9](#), [12](#), [13](#), [14](#), [15](#)
- Bologna, [16](#)
- boundTransform, [16](#)
- boundTransform_av_links, [20](#)
- boundTransform_link.gumbel, [20](#)
- boundTransform_link.identity, [21](#)
- boundTransform_link.invgumbel, [21](#)
- boundTransform_link.logis, [21](#)
- boundTransform_link.norm, [22](#)
- CAMS_data, [22](#)
- CDF (PDF), [56](#)
- clearsky_delta, [23](#), [27](#)
- clearsky_optimizer, [23](#)
- clearsky_optimizer_CLS, [24](#)
- clearsky_optimizer_delta, [25](#)
- clearsky_optimizer_RLS, [25](#)
- clearsky_outliers, [26](#)
- control_seasonalClearsky, [24](#), [27](#), [58](#), [92](#)
- control_solarHedging, [28](#), [74](#), [75](#)
- control_solarOption, [28](#), [74](#), [75](#), [108](#), [109](#), [111](#), [113–115](#), [117](#), [118](#)
- desscher, [29](#), [31](#), [33–35](#), [38](#), [39](#), [41](#), [42](#), [44](#), [46](#)
- desscherMixture, [30](#), [30](#), [33–35](#), [38](#), [39](#), [41](#), [42](#), [44](#), [46](#)
- detect_season, [31](#)
- dgumbel, [30](#), [31](#), [32](#), [34](#), [35](#), [38](#), [39](#), [41](#), [42](#), [44](#), [46](#)
- dinvgumbel, [30](#), [31](#), [33](#), [33](#), [35](#), [38](#), [39](#), [41](#), [42](#), [44](#), [46](#)
- dkumaraswamy, [30](#), [31](#), [33](#), [34](#), [34](#), [38](#), [39](#), [41](#), [42](#), [44](#), [46](#)
- dmix2norm, [35](#)
- dmixnorm, [30](#), [31](#), [33–35](#), [37](#), [39](#), [41](#), [42](#), [44](#), [46](#)
- dsnrm, [30](#), [31](#), [33–35](#), [38](#), [38](#), [41](#), [42](#), [44](#), [46](#)
- dsolarGHI, [30](#), [31](#), [33–35](#), [38](#), [39](#), [39](#), [42](#), [44](#), [46](#)
- dsolarK, [30](#), [31](#), [33–35](#), [38](#), [39](#), [41](#), [41](#), [44](#), [46](#)
- dsolarX, [30](#), [31](#), [33–35](#), [38](#), [39](#), [41](#), [42](#), [43](#), [44](#), [46](#)

- dsugeno, [30](#), [31](#), [33–35](#), [38](#), [39](#), [41](#), [42](#), [44](#), [44](#), [46](#)
- dtnorm, [30](#), [31](#), [33–35](#), [38](#), [39](#), [41](#), [42](#), [44](#), [45](#)
- GARCH_companion_matrix, [46](#)
- gaussianMixture, [47](#), [75](#)
- is_leap_year, [50](#)
- ks_test, [51](#), [88](#)
- ks_test_ts, [52](#)
- ks_ts_test (ks_test), [51](#)
- logLik_std.errors, [52](#)
- mix2norm_M_Yl_given_Yv, [53](#)
- mix2norm_probs_given_Yv, [53](#)
- mix2norm_S_Yl_given_Yv, [54](#)
- monthlyParams, [54](#), [75](#)
- number_of_day, [55](#)
- PDF, [56](#)
- pesscher (desscher), [29](#)
- pesscherMixture (desscherMixture), [30](#)
- pgumbel (dgumbel), [32](#)
- pinvgumbel (dinvgumbel), [33](#)
- PIT, [57](#)
- PIT_psolarGHI (PIT), [57](#)
- PIT_test, [57](#)
- pkumaraswamy (dkumaraswamy), [34](#)
- pmix2norm (dmix2norm), [35](#)
- pmixnorm (dmixnorm), [37](#)
- psnorm (dsnrm), [38](#)
- psolarGHI (dsolarGHI), [39](#)
- psolarK (dsolarK), [41](#)
- psolarX (dsolarX), [43](#)
- psugeno (dsugeno), [44](#)
- ptnorm (dtnorm), [45](#)
- qgumbel (dgumbel), [32](#)
- qinvgumbel (dinvgumbel), [33](#)
- qkumaraswamy (dkumaraswamy), [34](#)
- qmix2norm (dmix2norm), [35](#)
- qmixnorm (dmixnorm), [37](#)
- qsnorm (dsnrm), [38](#)
- qsolarGHI (dsolarGHI), [39](#)
- qsolarK (dsolarK), [41](#)
- qsolarX (dsolarX), [43](#)
- qtnorm (dtnorm), [45](#)
- Quantile (PDF), [56](#)
- rgumbel (dgumbel), [32](#)
- riccati_root, [57](#)
- rinvgumbel (dinvgumbel), [33](#)
- rkumaraswamy (dkumaraswamy), [34](#)
- rmixnorm (dmixnorm), [37](#)
- rsnorm (dsnrm), [38](#)
- rsolarGHI (dsolarGHI), [39](#)
- rsolarK (dsolarK), [41](#)
- rsolarX (dsolarX), [43](#)
- rtnorm (dtnorm), [45](#)
- seasonalClearsky, [24–26](#), [58](#), [81](#), [90](#)
- seasonalClearsky_spec, [24](#), [58](#), [60](#), [92](#)
- seasonalModel, [61](#), [81](#), [90](#)
- seasonalSolarFunctions, [58](#), [64](#)
- sGARCH, [71](#), [90](#)
- solarHedging_model, [73](#)
- solarHedging_scenarios, [74](#)
- solarMixture, [75](#), [91](#)
- solarMixture_VaR, [79](#)
- solarModel, [74](#), [79](#), [80](#), [103–105](#), [108](#), [109](#), [111](#), [113](#), [118](#), [121](#)
- solarModel_AIC_BIC, [84](#)
- solarModel_calibrate_theta, [84](#)
- solarModel_covariance, [85](#)
- solarModel_forecast, [85](#)
- solarModel_grid, [86](#)
- solarModel_match_params, [87](#)
- solarModel_PIT, [87](#), [88](#)
- solarModel_PIT_test, [88](#)
- solarModel_predict, [88](#)
- solarModel_predict_plot, [89](#)
- solarModel_QMLE, [89](#)
- solarModel_scenarios, [118](#)
- solarModel_spec, [80](#), [90](#), [104](#)
- solarModel_test_autocorr, [97](#)
- solarModel_test_distribution, [98](#)
- solarModel_test_forecast, [99](#)
- solarModel_test_LPD, [99](#)
- solarModel_test_pricing, [99](#)
- solarModel_tests, [96](#)
- solarModel_VaR, [100](#)
- solarMoment, [100](#)
- solarMoments, [74](#), [102](#), [109](#), [113](#), [117](#)
- solarMoments_conditional, [103](#)
- solarMoments_path, [104](#)
- solarMoments_unconditional, [105](#)
- solarOption, [105](#)
- solarOption_calibrate_theta, [108](#)
- solarOption_choquet, [108](#)
- solarOption_discount, [109](#)
- solarOption_greeks, [110](#)
- solarOption_historical, [111](#)
- solarOption_historical_bootstrap, [29](#)
- solarOption_index_greeks, [112](#)

`solarOption_lambda`, [112](#)
`solarOption_model`, [113](#)
`solarOption_moments`, [114](#)
`solarOption_payoff`, [115](#)
`solarOption_price`, [116](#)
`solarOption_pricing`, [117](#)
`solarOption_scenario`, [118](#)
`solarOptionPayoff`, [107](#)
`solarr::boundTransform`, [126](#)
`solarr::seasonalModel`, [58](#)
`solarScenario`, [74](#), [119](#)
`solarScenario_by`, [120](#)
`solarScenario_filter`, [121](#)
`solarScenario_plot`, [122](#)
`solarScenario_residuals`, [122](#)
`solarScenario_spec`, [123](#)
`solarScenario_VaR`, [125](#)
`solarSpectra`, [125](#)
`solarTransform`, [81](#), [90](#), [92](#), [126](#)
`SoRadPorfolio`, [128](#)
`spectralDistribution`, [129](#)
`stats::arima()`, [11](#)
`sugeno_bounds`, [129](#)

`VaR_test`, [130](#)
`VaR_viol`, [130](#)